

ЛАБОРАТОРНАЯ РАБОТА № 10

ДЕШИФРОВАНИЕ ШИФРА ПРОСТОЙ ПЕРЕСТАНОВКИ ПРИ ПОМОЩИ МЕТОДА БИГРАММ

Цель работы: знакомство с основами шифрования (расшифрования) данных методом перестановки. Освоение метода дешифрования данных с использованием биграмм.

Описание лабораторной работы

Описание метода шифрования. Перестановочные шифры используются с древних времен. Однако практически в любой современной симметричной криптосистеме можно найти элементы перестановок (например, в алгоритме *DES* — матрицы начальной и конечной перестановок). Приведем пример шифра перестановки. Выберем целое положительное число, скажем, $5(n)$; расположим числа от 1 до $5(n)$ в двухстрочной записи, в которой вторая строка — произвольная перестановка чисел верхней строки:

1	2	3	4	5
3	2	5	1	4

Эта конструкция носит название подстановки, а число 5 называется ее степенью.

Зашифруем фразу «СВЯЩЕННАЯ РИМСКАЯ ИМПЕРИЯ». В этой фразе 23 буквы. Дополним ее двумя произвольными буквами (например, Ъ, Э) до ближайшего числа, кратного 5, т.е. до 25. Выпишем эту дополненную фразу без пропусков, разбивая ее на группы по пять символов в соответствии со степенью подстановки:

СВЯЩЕ ННАЯР ИМСКА ЯИМПЕ РИЯЪЭ

Буквы каждой группы переставим в соответствии с указанной двухстрочной записью по следующему правилу: первая буква встает на третье место, вторая — на второе, третья — на пятое, четвертая — на первое и пятая — на четвертое. Полученный текст выписывается без пропусков:

ЩВСЕЯЯННРАКМИАСПИЯЕМЬИРЭЯ

При расшифровании текст разбивается на группы по пять букв и буквы переставляются в обратном порядке: первая на четвертое место, вторая на второе, третья на первое, четвертая на пятое и пятая на третье. Ключом шифра является выбранная степень подстановки

(в нашем случае число 5) и порядок расположения чисел в нижнем ряду двухстрочной записи.

В соответствии с методом математической индукции можно легко убедиться в том, что существует $1 \times 2 \times 3 \times \dots \times n(n!)$ вариантов заполнения нижней строки двухстрочной записи. Таким образом, число различных преобразований шифра перестановки, предназначенного для шифрования сообщений длины n , меньше либо равно $n!$ (заметим, что в это число входит и вариант преобразования, оставляющий все символы на своих местах).

Описание метода дешифрования. Сообщения, как бы сложны они ни были, можно представить в виде последовательности знаков. Эти знаки берутся из заранее фиксированного набора, например, кириллического алфавита или палитры цветов (красный, зеленый, синий). Разные знаки встречаются в сообщениях с разной частотой. Поэтому количество информации, передаваемой разными знаками, может быть разным. По известной формуле К. Шеннона количество информации определяется средним числом возможных вопросов с ответами ДА и НЕТ для того, чтобы угадать следующий символ сообщения.

Эффективное кодирование базируется на *основной теореме Шеннона для канала без помех*, суть которой сводится к следующему.

Суть теоремы Шеннона

Сообщения, составленные из букв некоторого алфавита, можно закодировать так, что среднее число двоичных символов на букву сколь угодно близко к энтропии источника этих сообщений, но не меньше этой величины.

Если буквы в тексте следуют независимо друг от друга, то среднее количество информации в сообщении, приходящееся на один знак, равно:

$$H = P_i \log_2 \frac{1}{P_i},$$

где P_i — частота появления символа i .

Отметим три особенности такого определения количества информации.

1. Оно абсолютно не связано с семантикой, смыслом сообщения, и им можно пользоваться, даже когда точный смысл неясен.
2. В нем предполагается независимость вероятности появления знаков от их предыстории.
3. Заранее известна знаковая система, в которой передается сообщение, т.е. язык, способ кодирования.

В каких единицах выражается значение количества информации по Шеннону? Точнее всего ответ на этот вопрос дает теорема кодирования, утверждающая, что любое сообщение можно закодировать символами 0 и 1 так, что полученная длина сообщения будет сколь угодно близка сверху к H . Эта теорема позволяет назвать и единицу информации — *бит*.

Каждый, кто использовал, работая на персональном компьютере, архиваторы, знает, сколь эффективно они сжимают текстовые файлы, ничего при этом не теряя. Их работа лучшим образом демонстрирует теорему Шеннона в действии. Так как для русского текста, переданного лишь прописными буквами, $H = 4,43$, и это означает, что, в принципе, в русском алфавите можно было бы обойтись лишь 22 буквами или на 45% сократить длину файлов в кодировке *ASCII*. Таким образом, сообщения занимают места больше, чем это необходимо. Это явление называют избыточностью языка, благодаря чему искажения отдельных символов сообщения зачастую не разрушают содержания, что случилось бы при отсутствии избыточности. Заметьте, у компьютера наиболее часто встречаемые символы *ETOANIRSHDLU* (даны в порядке убывания частот в английском языке) вынесены в центр клавиатуры, чтобы при наборе текстов движение пальцев было бы минимальным. Это расположение клавиш было предложено изобретателем линотипа Оттомаром Мергенталером, который использовал избыточность языка для облегчения работы. Утверждение, что вероятность появления символа в связном тексте не зависит от его предыстории, неверно и статистически, и лингвистически. Уже давно филологи заметили, что обычно за согласной буквой следует гласная, а за гласной согласная. Поэтому в конце XIX в. петербургский математик А.А. Марков предложил рассматривать текст как цепочку символов, где вероятность появления буквы зависит от предыдущей и только от нее. Таким образом, он стал рассматривать не вероятности P_j появления в сообщении знака j , а вероятности P_{ij} появления знака j при условии, что перед ним стоит знак i . Теория марковских цепей оказалась чрезвычайно продуктивной для криптографии, и к отдельным ее применениям мы будем возвращаться позже. Пока же достаточно отметить, что первое свое опробование она имела при анализе текстов «Евгения Онегина» самим Андреем Андреевичем Марковым. Объем информации в одном символе марковской цепи определяется следующей формулой:

$$H = P_i \left(P_{ij} \operatorname{Log}_2 \frac{1}{P_{ij}} \right).$$

В этом случае нет противоречия с требованием независимости знаков, так как знаком здесь считается не отдельный символ, а биграмма. В приложении приведена таблица вероятности встречаемости биграмм в русском техническом тексте. Вероятности представлены десятью классами от 0 до 9 в порядке возрастания и образуют по средним значениям геометрическую прогрессию. Справа в этой таблице даны вероятности встречаемости отдельных символов. Так, из нее следует, что биграмма АЙ встречается довольно часто (класс 7), а биграмма ЙА почти совсем не попадает (класс 0). Среднее количество информации, приходящееся на один символ, определяемое по этой таблице, равно 3,5 бит.

Описанное свойство зависимости буквы в тексте от предыдущей называется марковостью первого порядка, а независимость букв друг от друга — марковостью нулевого порядка. Естественно, что можно рассматривать также и марковости высших порядков, например второго, когда буква зависит от двух предыдущих. Для того чтобы оценить порядок марковости в связном тексте, можно провести случайное моделирование, используя сначала вероятности отдельных букв, потом биграмм, триграмм и т.д. Очевидно, что увеличение порядка марковости повышает схожесть отрывка случайного текста с естественным. Повышение порядка марковости позволяет доуточнить объем информации в сообщениях, но это очень скользкая тема и есть масса разных точек зрения на нее. Действительно, вводя понятие шенноновской меры информации, мы отказались от понятия смысла, который связывает символы в слоги, слоги в слова, слова в предложения, а предложения в сообщение. Практически нет разницы, как сказать ребенку: «Нужно есть кашу!» или «Надо есть кашу!», а вот шенноновский подход эти сообщения считает различными. Поэтому оценка объема информации, содержащейся в сообщении и полученной по приведенным формулам, явно завышена.

Возьмем пример шифра двойной перестановки. Пусть имеется шифровка **АЗЮЖЕ СШГТООИПЕР**, которая так укладывается в таблицу 4×4:

	1	2	3	4
1	А	З	Ю	Ж
2	Е		С	Ш
3	Г	Т	О	О
4	И	П	Е	Р

Рассматривая маловероятные сочетания букв, легко найти истинную последовательность столбцов. Так, сочетание ГТ в третьей строке шифровки указывает на то, что после первого столбца вряд ли следует второй столбец. Рассчитаем статистически, какой столбец скорее всего следует за первым. Для этого воспользуемся таблицей логарифмов вероятностей биграмм русского текста, приведенной в приложении. Вероятность следования одного столбца за другим равна сумме коэффициентов биграмм в строках этих столбцов. Для коэффициентов следования за первым столбцом второй, третий и четвертый имеем выражения:

$$\begin{aligned}k(1-2) &= k(A3) + k(E) + k(ГТ) + k(ИП) = 7 + 9 + 0 + 5 = 21; \\k(1-3) &= k(AЮ) + k(ЕС) + k(ГО) + k(ИЕ) = 6 + 8 + 8 + 8 = 30; \\k(1-4) &= k(AЖ) + k(ЕШ) + k(ГО) + k(ЯР) = 7 + 5 + 8 + 7 = 27.\end{aligned}$$

В нашем случае наиболее вероятно, что после столбца 1 следует столбец 3. Для такой небольшой таблицы шифрования, которую имеем, можно перебрать все варианты перестановок — их всего лишь 24. В случае большого числа столбцов целесообразно оценить вероятности пар сочетаний разных столбцов и решить оптимизационную задачу, которая укажет перестановку столбцов, дающую фрагменты естественного текста с большей вероятностью. В нашем случае наилучший результат достигается при расстановке столбцов (2 4 1 3), что примерно вдвое по вероятностной оценке достовернее ближайшей к ней по вероятности расстановки (4 1 3 2). После того как столбцы шифровки расставлены, не составит труда правильно расставить и ее строки по смыслу фрагментов текста:

	2	4	1	3
1	3	Ж	А	Ю
2		Ш	Е	С
3	Т	О	Г	О
4	П	Р	И	Е

Текст в ней уже читается и, расставив строки в порядке (4 1 2 3), получим расшифровку **ПРИЕЗЖАЮ ШЕСТОГО**.

Можно привести еще один пример расшифровки с использованием биграмм. Зашифруем текст методом перестановки:

ТЕОРИЯ ИНФОРМАЦИИ

Дополним текст до полной матрицы при помощи некоторых случайных букв, в нашем случае «ПОР».

Т	Е	О	Р
И	Я		И
Н	Ф	О	Р
М	А	Ц	И
И	П	О	Р

Ключ: 1 2 3 4
 4 2 1 3

Получим:

Зашифрованный текст: ОЕРТ ЯИИОФРНЦАИМОПРИ

О	Е	Р	Т
	Я	И	И
О	Ф	Р	Н
Ц	А	И	М
О	П	Р	И

Для упрощения задачи предположим, что длина ключа известна. Далее, как и в предыдущем примере, необходимо оценить вероятности биграмм в столбцах и решить оптимизационную задачу.

Например, для первого столбца:

$$k(1-2) = k(ОЕ) + k(Я) + k(ОФ) + k(ЦА) + k(ОП) = \\ = 6 + 0 + 2 + 5 + 4 = 17;$$

$$k(1-3) = k(ОР) + k(И) + k(ОР) + k(ЦИ) + k(ОР) = \\ = 8 + 8 + 8 + 7 + 8 = 39;$$

$$k(1-4) = k(ОТ) + k(И) + k(ОН) + k(ЦМ) + k(ОИ) = \\ = 8 + 8 + 7 + 0 + 6 = 29.$$

Таким образом, самый большой коэффициент следования у столбцов 1—3, проводя дальнейшие вычисления, несложно будет определить и весь ключ.

Применение данного метода достаточно эффективно при дешифровании осмысленных сообщений, так как при шифровании методом перестановки вероятностная характеристика символов в осмысленном сообщении сохраняется, что существенно упрощает взлом.

Описание демонстрационной программы. Программный модуль **снс.exe** используется для автоматизации процесса шифрования (расшифрования) методом перестановки и вскрытия шифротекстов методом биграмм. Главное окно программы и окно дешифрования представлены на рис. 3.2.

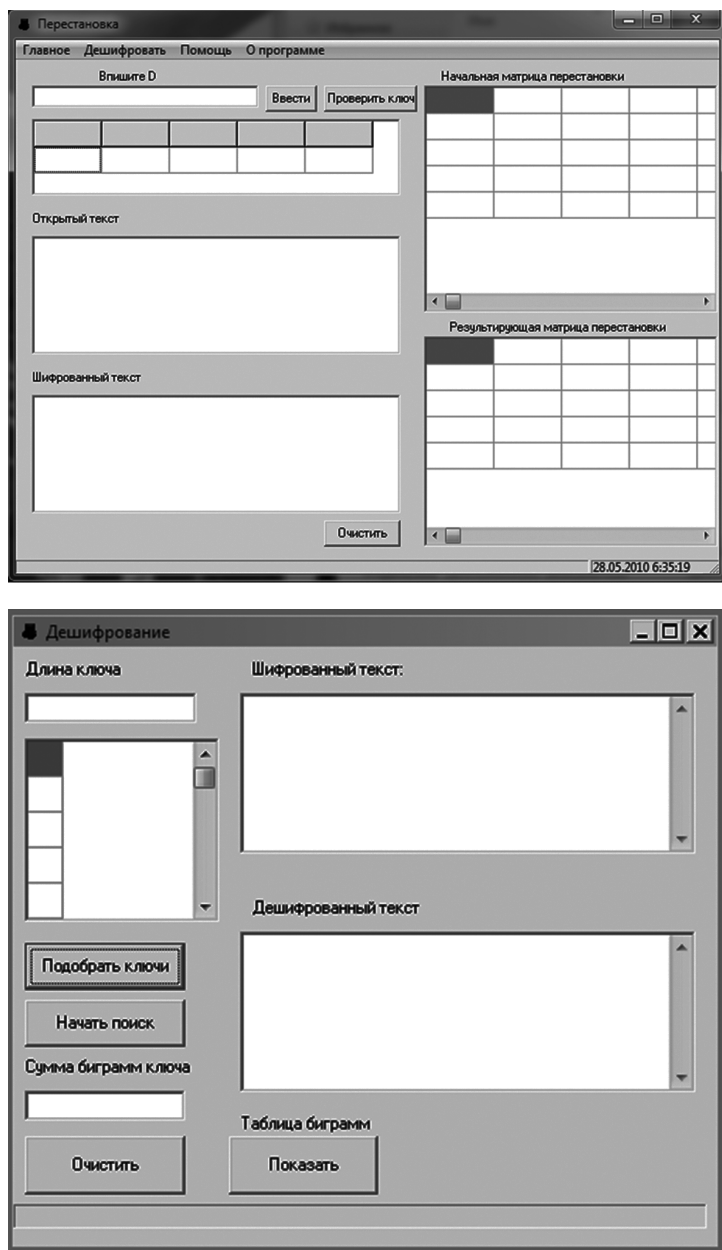


Рис. 3.2. Главное окно программы и окно дешифрования

Дополнительные сведения о программе и авторах программной реализации представлены на рис. 3.3.

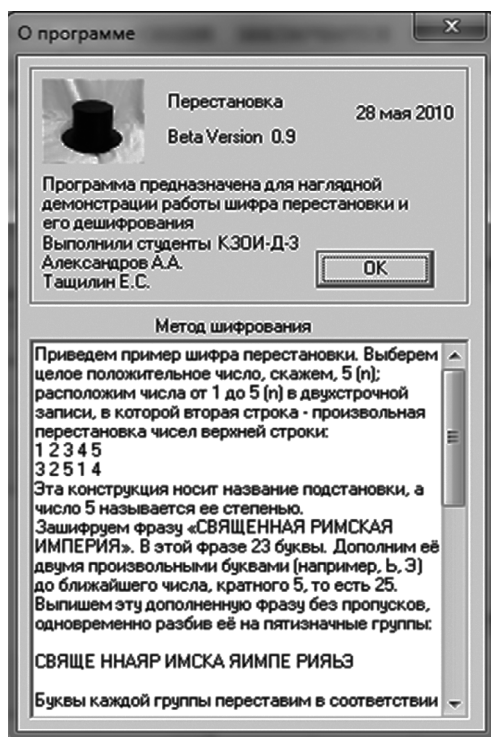


Рис. 3.3. Дополнительные сведения о программе и авторах программной реализации

Задание

Для выполнения лабораторной работы на компьютере необходимо установить файл *cns.exe*.

Запустить программу *cns.exe*, предназначенную для демонстрации шифрования (расшифрования) данных методом перестановки и освоения метода дешифрования данных с использованием биграмм.

Примечание. Для обеспечения корректной функциональной работы программы на операционной системе Windows 10 при запуске *cns.exe* необходимо включить режим совместимости.

1. Зашифровать (расшифровать) вводимый с клавиатуры текст и тексты, открываемые из файлов.

Внимание! Предварительно просмотрите приложенные к программе тестовые примеры о формате и структуре шифруемых файлов.

Привести в отчете экранные формы выполняемых действий.

2. Произвести попытку дешифрования данных, зашифрованных в п. 1 (*если попытка вскрытия криптограмм не удалась, сделать выводы о причинах неудачи*). Привести в отчете экранные формы.

3. Добиться правильного дешифрования зашифрованного текста (*подбором длины и содержания исходного текста, длины ключа шифрования*). Приложить экранные формы, сделать выводы об особенностях исследуемого метода вскрытия криптограмм.

4. Сохранить в отчете экранные формы, демонстрирующие процесс шифрования, расшифрования и дешифрования.

5. Включить в отчет о лабораторной работе ответы на контрольные вопросы, выбранные в соответствии с номером варианта из табл. 3.2

Таблица 3.2

Номер варианта	Контрольные вопросы
1, 5, 7, 3, 9, 18, 28	В чем заключается описанный метод вскрытия криптограмм?
2, 4, 6, 8, 20, 22, 24, 26, 30	В чем заключается метод шифрования (расшифрования) с использованием перестановок? Какие перестановочные методы шифрования вы знаете?
11, 13, 15, 10, 17, 19, 27	Приведите примеры использования алгоритма перестановки в современных симметричных криптосистемах
12, 14, 16, 21, 23, 25, 29	Какие требования к исходным текстам и длинам ключей шифрования обеспечат максимальный эффект для использования изученного метода дешифрования?

Приложение 3.1

Таблица коэффициентов встречаемости биграмм в тексте

	АБВГДЕЖЗИЙКЛМНОПРСТУФХЦШТЬЬЪЭЮЯ	Процент
А	278677774777883767826677550000679	71
Б	711016226056357275070541055722035	13
В	805048037167568466660301300820048	38
Г	601165006045448070060012000000004	10
Д	816348107047178465271333400640457	25
Е	556786664778896588933656560011559	73
Ж	600067217050271012130000000020002	8
З	846264116155667150060021002620046	14
И	667668577776885578815777630100679	64
И	003030000036540006600012300000008	11
К	815116527127058076670060100000007	29
Л	841218618044167003363003110680786	31
М	757228017044768513161000000730068	32
Н	903368119060178005765253000850467	50
О	288886776878876788832567650015259	89
П	700008047036148494562010000453044	28
Р	916448608052668426673542420740167	48
С	646257207078668756963515500561387	43
Т	827148008064569388460004021780158	60
У	344667653365560677715506360000748	19
Ф	600005006002206040354000000100002	4
Х	433004003011056053130020001000008	6
Ц	506006007000003000040000000500005	6
Ч	701008007061062010730001300130004	12
Ш	500006007033034030340000000040005	3
Щ	600007006000020020040000000040101	3
Ъ	00000400000000000000000000000032	1
Ы	147357151755621555600705410100018	18
Ь	010003071060471006400001610000628	12
Э	004001000265210201704300000000001	4
Ю	050020120410000003700006170010307	6
Я	015256250223650144700443040000649	18
	78978758838689989877678511218260	138

ЛАБОРАТОРНАЯ РАБОТА № 11

СЕТЬ ФЕЙСТЕЛЯ

Цель работы: изучение принципа работы сети Фейстеля.

Описание лабораторной работы. В 1971 году Хорст Фейстель (*Horst Feistel*) запатентовал два устройства для реализации различных алгоритмов шифрования, получившие название «Люцифер» (*Lucifer*). Одно из устройств использовало конструкцию, впоследствии названную «сетью Фейстеля» (*Feistel cipher, Feistel network*). Работа над созданием новых криптосистем велась Фейстелем в стенах IBM вместе с Доном Копперсмитом (*Don Coppersmith*). Проект «Люцифер» был скорее экспериментальным, но стал базисом для алгоритма *DES (Data Encryption Standard)*. В 1973 году Хорст Фейстель в журнале *Scientific American* опубликовал статью «Криптография и Компьютерная безопасность» (*Cryptography and Computer Privacy*), в которой раскрыл ряд важных аспектов шифрования и привел описание первой версии проекта «Люцифер».

Описание алгоритма. Сеть Фейстеля подразумевает разбиение обрабатываемого блока данных на несколько субблоков (чаще всего — на два), один из которых обрабатывается некоей функцией $f(\alpha)$ и накладывается на один или несколько остальных субблоков. На рисунке 3.4 приведена наиболее часто встречающаяся структура алгоритмов на основе сети Фейстеля.

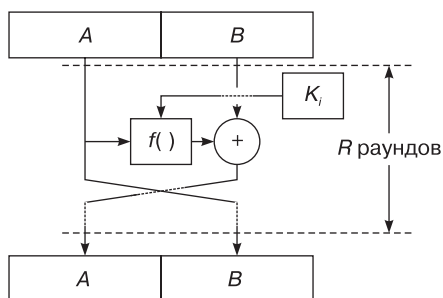


Рис. 3.4. Сеть Фейстеля

Блок открытого текста делится на две равные части A — левая (L) и B — правая (R), в каждом раунде i ($i = 1 \dots n$ — номер раунда) вычисляется

$$\begin{aligned} L_i &= R_{i-1} \oplus f(L_{i-1}, K_{i-1}); \\ R_{i-1} &= L_{i-1}, \end{aligned}$$

где $f()$ — некая функция, а K_{i-1} — ключ i -го раунда.

Результатом выполнения n раундов является (L_n, R_n) , но обычно в n -ом раунде перестановка L_n и R_n не производится, что позволяет использовать ту же процедуру и для расшифрования, просто инвертировав порядок использования раундовой ключевой информации:

$$\begin{aligned} L_{i-1} &= R_i \oplus f(L_i, K_{i-1}); \\ R_{i-1} &= L_i. \end{aligned}$$

Одно из преимуществ такой модели — обратимость алгоритма независимо от используемой функции f , причем она может быть сколь угодно сложной.

Надежность алгоритма. Сети Фейстеля были широко изучены криптографами в силу их обширного распространения. В 1988 году Майкл Люби (*Michael Luby*) и Чарльз Ракофф (*Charles Rackoff*) провели исследования сети Фейстеля и доказали, что если раундовая функция является криптостойкой псевдослучайной и используемые ключи независимы в каждом раунде, то трех раундов будет достаточно для того, чтобы блочный шифр являлся псевдослучайной перестановкой.

Иногда сеть Фейстеля в западной литературе называют *Luby — Rackoff block cipher* в честь Люби и Ракоффа, которые проделали большой объем теоретических исследований в этой области.

В дальнейшем, в 1997 г., Мони Наор (*Moni Naor*) и Омер Рейнголд (*Omer Reingold*) предложили упрощенный вариант конструкции Люби — Ракоффа из четырех раундов, где в качестве первого и последнего раунда используются две попарно независимые перестановки. Два средних раунда конструкции Наора — Рейнголда идентичны раундам в конструкции Люби — Ракоффа.

Во многих блочных шифрах на основе сети Фейстеля были найдены те или иные уязвимости, однако в ряде случаев эти уязвимости являются чисто теоретическими и при нынешней производительности компьютеров использовать их на практике для взлома невозможно.

Симметричные криптоалгоритмы, использующие сеть Фейстеля. Так как большинство блочных шифров создано на основе сетей Фейстеля, коротко упомянем некоторые из них.

DES

DES работает с 64-битовым блоком открытого текста. После первоначальной перестановки блок разбивается на правую и левую половины длиной по 32 бита. Затем выполняются 16 этапов одинаковых действий, определяемых функцией f , в которых данные объединяются с ключом. После шестнадцатого этапа правая и левая половины объединяются и алгоритм завершается заключительной перестановкой (обратной по отношению к первоначальной).

На каждом этапе биты ключа сдвигаются и затем из 56 битов ключа выбираются 48 битов для цикловых ключей. Правая половина данных увеличивается до 48 битов с помощью перестановки с расширением, объединяется посредством XOR с 48 битами смещенного и переставленного ключа, проходит через восемь *S*-блоков, образуя 32 новых бита, и переставляется снова. Эти четыре операции и выполняются функцией *f*. Затем результат функции *f* объединяется с левой половиной с помощью другого XOR. В итоге этих действий появляется новая правая половина, а старая правая половина становится новой левой. Эти действия повторяются 16 раз, образуя 16 циклов.

FEAL

В качестве входа процесса шифрования используется 64-битовый блок открытого текста. Сначала блок данных подвергается операции XOR с 64 битами ключа. Затем блок данных расщепляется на левую и правую половины. Объединение левой и правой половин с помощью XOR образует новую правую половину. Левая половина и новая правая половина проходят через *n* этапов (первоначально четыре). На каждом этапе правая половина объединяется с помощью функции *f* с шестнадцатью битами ключа, и с помощью XOR — с левой половиной, создавая новую правую половину. Исходная правая половина (на начало этапа) становится новой левой половиной. После *n* этапов (левая и правая половины не переставляются после *n*-го этапа) левая половина снова объединяется с помощью XOR с правой половиной, образуя новую правую половину, затем левая и правая соединяются вместе в 64-битовое целое. Блок данных объединяется с помощью XOR с другими 64 битами ключа, и алгоритм завершается.

Функция *f* использует 32 бита данных и 16 битов ключа и смешивает их вместе. Сначала блок данных разбивается на 8-битовые подблоки, которые затем объединяются с помощью XOR и меняются местами.

RC2

RC2 — это шифр с 64-битовым блоком и переменной длиной ключа, разработанный как альтернатива DES. В соответствии с утверждениями компании *RSA Data Security* программные реализации RC2 в три раза быстрее DES. Алгоритм может использовать ключ переменной длины, от 0 байтов до максимальной длины строки, поддерживаемой компьютерной системой, причем скорость шифрования не зависит от размера ключа. Этот ключ предварительно используется для заполнения 128-байтовой таблицы, зависящей от ключа. RC2 не использует *S*-блоков, здесь используются две операции — «смешивание» и «перемешивание» (*mix* и *mash*), для каждого этапа выбирается одна из них.

RC2 не является итеративным блочным шифром. Это предполагает, что RC2 более устойчив к дифференциальному и линейному криптоанализу, чем другие блочные шифры, безопасность которых опирается на копировании схемы DES.

ГОСТ-28147—89

ГОСТ является 64-битовым алгоритмом с 256-битовым ключом. ГОСТ также использует дополнительный ключ в виде восьми различных S -блоков, функционирование которых рассматривается ниже. В процессе шифрования на 32 этапах последовательно выполняется алгоритм шифрования Фейстеля.

Для шифрования текст сначала разбивается на левую половину L и правую половину R . На этапе i алгоритма ГОСТ используется подключ K_i и выполняются следующие действия:

$$\begin{aligned} L_i &= R_{i-1}; \\ R_i &= L_{i-1}f(R_{i-1}, K_i). \end{aligned}$$

Функция f проста. Сначала правая половина и i -й подключ складываются по модулю 2^{32} . Результат разбивается на восемь четырехбитовых подблоков, каждый из которых поступает на вход своего S -блока. ГОСТ использует восемь различных S -блоков, первые четыре бита попадают в первый S -блок, вторые четыре бита — во второй S -блок, и т.д. Каждый S -блок представляет собой перестановку чисел от 0 до 15.

В этом случае, если на входе S -блока 0, то на выходе 7. Если на входе 1, на выходе 10, и т.д. Все восемь S -блоков в ГОСТ-28147—89 различны, они фактически являются дополнительным ключевым материалом. S -блоки должны храниться в секрете.

Выходы всех восьми S -блоков объединяются в 32-битовое слово, затем все слово циклически сдвигается влево на 11 битов. Наконец результат объединяется с помощью XOR с левой половиной и получается новая правая половина, а правая половина становится новой левой половиной. Количество таких циклов — 32.

Генерация подключей проста. 256-битовый ключ разбивается на восемь 32-битовых блоков: $k_1, k_2, \dots, k_8$. На каждом этапе используется свой подключ. Расшифрование выполняется также как и шифрование, но инвертируется порядок подключей k_i .

ГОСТ-28147—89 не определяет способ генерации S -блоков, говорится только, что блоки должны быть предоставлены каким-то образом. Это породило домыслы о том, что советский производитель может поставлять хорошие S -блоки «хорошим» организациям и плохие S -блоки тем организациям, которых производитель собирается надуть,

вследствие чего неофициальные переговоры с российским производителем микросхем ГОСТ выявили другую альтернативу. Производитель создает перестановки S -блока самостоятельно с помощью генератора случайных чисел.

Программная реализация сети Фейстеля

```
/* функция преобразования подблока по ключу (зависит
от конкретного алгоритма)
subblock — преобразуемый подблок
key — ключ
возвращаемое значение — преобразованный блок*/
int f (int subblock, int key);
/* Шифрование открытого текста
left — левый входной подблок
right — правый входной подблок
* key — массив ключей (по ключу на раунд)
rounds — количество раундов*/
void crypt (int *left, int *right, int rounds, int *key)
{
    int i, temp;
    for (i = 0; i < rounds; i++)
    {
        temp = *right ^ f (*left, key[i]);
        *right = *left;
        *left = temp;
    }
}
/* Расшифрование текста
left — левый зашифрованный подблок
right — правый зашифрованный подблок*/
void decrypt (int *left, int *right, int rounds, int *key)
{
    int i, temp;
    for (i = rounds - 1; i >= 0; i--)
    {
        temp = *left ^ f (*right, key [i]);
        *left = *right;
        *right = temp;
    }
}
```

Описание программного интерфейса. Главное окно программы представлено на рис. 3.5.

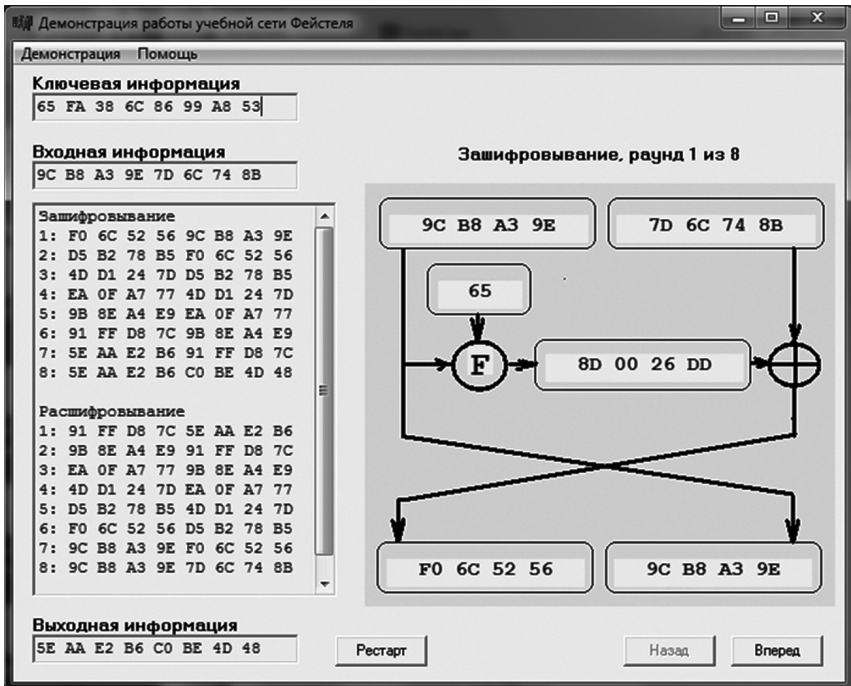


Рис. 3.5. Главное окно программы

Демонстрация

Рестарт — генерация случайного ключа (64 бит) и входной информации (64 бит).

Шаг вперед — переход к следующему раунду.

Шаг назад — переход к предыдущему раунду.

Выход — выход из программы.

Для удобства работы с программой основные функции продублированы кнопками в основном окне программы.

Помощь

Справка — содержит теоретическую информацию об алгоритме шифрования сеть Фейстеля и об использовании программы (рис. 3.6). Информация о разработчике представлена на рис. 3.7.

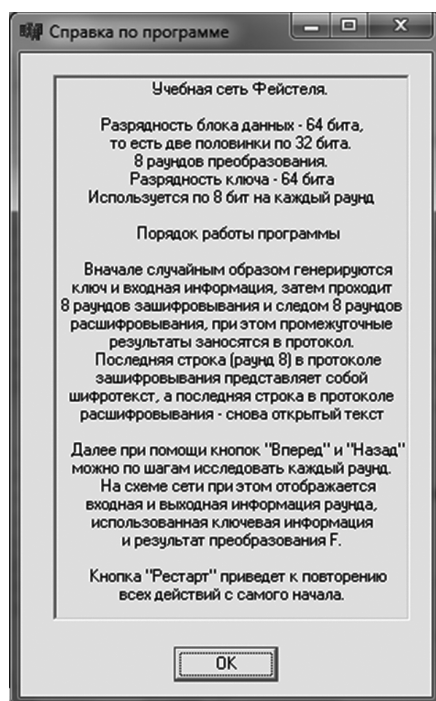


Рис. 3.6. Справочная информация

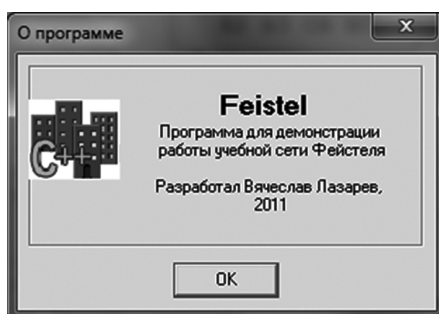


Рис. 3.7. Информация о разработчике

Поле «**Ключевая информация**» содержит случайно сгенерированный ключ.

Поле «**Входная информация**» содержит случайно сгенерированную информацию длиной 64 бита.

Поле **«Выходная информация»** содержит зашифрованную информацию.

Поле **«Зашифровывание»** отображает процесс шифрования, состоящий из восьми последовательных раундов.

Порядок выполнения программы

Нажать кнопку **«Рестарт»**, далее, используя клавиши вперед (назад), ознакомиться с процессом шифрования входной информации.

Задание

Для выполнения лабораторной работы на компьютере необходимо установить программу *Feistei.exe*. Запустить программу *Feistei.exe*, используемую для демонстрации работы сети Фейстеля.

1. Чтобы начать работу с программой, необходимо в Меню **«Демонстрация»** открыть вкладку **«Рестарт»** или нажать кнопку **«Рестарт»**.

2. С помощью переключателя **«Шаг вперед»** или кнопки **«Вперед»** можно последовательно наблюдать, каким образом шифруется на каждом цикле преобразования сети Фейстеля блок исходного текста.

3. Сохранить в отчете экранные формы, демонстрирующие процесс пошаговой работы сети Фейстеля при шифровании исходного текста.

4. Произвести вручную процесс шифрования в одном цикле и убедиться в том, что результаты, демонстрируемые программой, совпадают с произведенными расчетами. Сделать выводы по проделанной работе.

5. Включить в отчет о лабораторной работе ответы на контрольные вопросы, выбранные в соответствии с номером варианта из табл. 3.3.

Таблица 3.3

Номер варианта	Контрольные вопросы
1, 5, 7, 3, 9, 18, 28	В каких современных симметричных системах шифрования используется сеть Фейстеля?
2, 4, 6, 8, 20, 22, 24, 26, 30	В чем отличие сбалансированной сети Фейстеля от несбалансированной сети Фейстеля? В каких блочных криптосистемах используется сбалансированная сеть?
11, 13, 15, 10, 17, 19, 27	В каких современных симметричных системах шифрования не используется сеть Фейстеля? Какие механизмы шифрования используются в этих криптографических системах?
12, 14, 16, 21, 23, 25, 29	Какой длины используются блоки для шифрования и цикловые ключи в блочных криптосистемах DES, FIAL, Blowfish, ГОСТ-28147—89?

ЛАБОРАТОРНАЯ РАБОТА № 12

РЕГИСТРЫ СДВИГА С ЛИНЕЙНОЙ ОБРАТНОЙ СВЯЗЬЮ КАК ГЕНЕРАТОРЫ ПСЕВДОСЛУЧАЙНЫХ ЧИСЕЛ

Цель работы: изучение принципа работы генератора псевдослучайных последовательностей, основанного на регистре сдвига с линейной обратной связью.

Описание лабораторной работы

Общие сведения о регистрах сдвига с линейной обратной связью. Последовательности регистров сдвига используются как в криптографии, так и в теории кодирования. Их теория прекрасно проработана, потоковые шифры на базе регистров сдвига являлись рабочей лошадкой военной криптографии задолго до появления электроники.

*Регистр сдвига с обратной связью (далее **PcCcOC**) состоит из двух частей: регистра сдвига и функции обратной связи (рис. 3.8).* Регистр сдвига представляет собой последовательность битов. Количество битов определяется *длиной сдвигового регистра*, если длина равна n битам, то регистр называется *n -битовым сдвиговым регистром*. Всякий раз, когда нужно извлечь бит, все биты сдвигового регистра сдвигаются вправо на одну позицию. Новый крайний левый бит является функцией всех остальных битов регистра. На выходе сдвигового регистра оказывается один, обычно младший значащий, бит. *Периодом сдвигового регистра* называется длина получаемой последовательности до начала ее повторения.

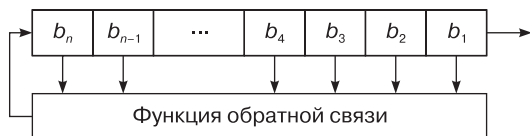


Рис. 3.8. Регистр сдвига с обратной связью

Регистры сдвига очень быстро нашли применение в потоковых шифрах, так как они легко реализовывались с помощью цифровой аппаратуры. В 1965 году Эрнст Селмер (*Ernst Selmer*), главный криптограф норвежского правительства, разработал теорию последовательности регистров сдвига. Соломон Голомб (*Solomon Golomb*), математик NSA, написал книгу, излагающие некоторые свои результаты и результаты Селмера. Простейшим видом регистра сдвига с обратной связью является *регистр сдвига с линейной обратной связью (linear feedback shift register, далее **LFSR** или **PcCcЛОС**)*. Обратная связь таких регистров

представляет собой просто XOR (сложение по модулю два) некоторых битов регистра, перечень этих битов называется отводной последовательностью (*tap sequence*). Иногда такой регистр называется конфигурацией Фибоначи. Из-за простоты последовательности обратной связи для анализа РгСсЛОС можно использовать довольно развитую математическую теорию. Проанализировав получаемые выходные последовательности, можно убедиться в том, что эти последовательности достаточно случайны, чтобы быть безопасными. РгСсЛОС чаще других сдвиговых регистров используются в криптографии.

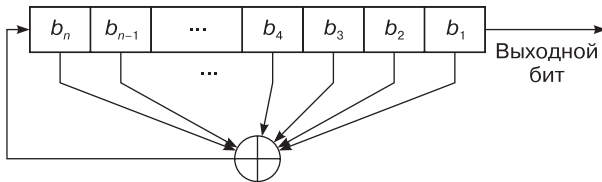


Рис. 3.9. РгСсЛОС Фибоначи

В общем случае n -битовый РгСсЛОС может находиться в одном из $N = 2^n - 1$ внутренних состояний. Это означает, что теоретически такой регистр может генерировать псевдослучайную последовательность с периодом $T = 2^n - 1$ битов. (Число внутренних состояний и период равны $N = T_{\max} = 2^n - 1$, потому что заполнение РгСсЛОС нулями приведет к тому, что сдвиговый регистр будет выдавать бесконечную последовательность нулей, что абсолютно бесполезно.) Только при определенных отводных последовательностях РгСсЛОС циклически пройдет через все $2^n - 1$ внутренних состояний, такие РгСсЛОС являются **РгСсЛОС с максимальным периодом**. Получившийся результат называется ***М-последовательностью***.

Пример. На рисунке 3.10 показан четырехбитовый РгСсЛОС с отводом от первого и четвертого битов. Если его проинициализировать значением 1111, то до повторения регистр будет принимать следующие внутренние состояния (табл. 3.4).

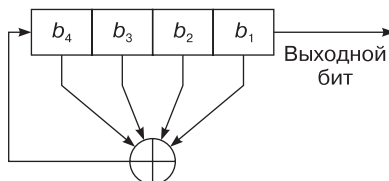


Рис. 3.10. Четырехбитовый РгСсЛОС с отводом от первого и четвертого битов

Таблица 3.4

Номер такта сдвига (внутреннего состояния)	Состояние регистров				Выходной бит
	T_1	T_2	T_3	T_4	
Инициальное значение	1	1	1	1	—
1	0	1	1	1	1
2	1	0	1	1	1
3	0	1	0	1	1
4	1	0	1	0	0
5	1	1	0	1	1
6	0	1	1	0	0
7	0	0	1	1	1
8	1	0	0	1	1
9	0	1	0	0	0
10	0	0	1	0	0
11	0	0	0	1	1
12	1	0	0	0	0
13	1	1	0	0	0
14	1	1	1	0	0
15 (возврат в инициальное состояние)	1	1	1	1	1
16 (повтор состояний)	0	1	1	1	1

Выходной последовательностью будет строка младших значащих битов: 111101011001000 с периодом $T = 15$, общее число возможных внутренних состояний (кроме нулевого) $N = 2^4 - 1 = 16 - 1 = 15 = T_{\max}$, следовательно, выходная последовательность — M -последовательность.

Для того чтобы конкретный РгСсЛОС имел максимальный период, многочлен, образованный из отводной последовательности и константы 1, должен быть примитивным по модулю 2. Многочлен представляется в виде суммы степеней, например многочлен степени n представляется так:

$$a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x^1 + a_0 x^0 = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0,$$

где $a_i = \{0, 1\}$ для $i = 1 \dots n$, а x^i — указывает разряд.

Степень многочлена является длиной сдвигового регистра. Примитивный многочлен степени n — это неприводимый многочлен, который является делителем $x^{2n-1} + 1$, но не является делителем $x^d + 1$ для всех d , являющихся делителями $2^n - 1$.

В общем случае не существует простого способа генерировать примитивные многочлены данной степени по модулю 2. Проще всего вы-

бирать многочлен случайным образом и проверять, не является ли он примитивным. Это нелегко и чем-то похоже на проверку, не является ли простым случайно выбранное число — но многие математические пакеты программ умеют решать такую задачу.

Некоторые, но, конечно же не все, многочлены различных степеней, примитивные по модулю 2, приведены далее. Например, запись (32, 7, 5, 3, 2, 1, 0) означает, что следующий многочлен примитивен по модулю 2: $x^{32} + x^7 + x^5 + x^3 + x^2 + x + 1$.

Это можно легко обобщить для ПрСсЛОС с максимальным периодом. Первым числом является длина ПрСсЛОС. Последнее число всегда равно 0, и его можно опустить. Все числа, за исключением 0, задают отводную последовательность, отсчитываемую от левого края сдвигового регистра. Другими словами, члены многочлена с меньшей степенью соответствуют позициям ближе к правому краю регистра.

Продолжая пример, запись (32, 7, 5, 3, 2, 1, 0) означает, что для взятого 32-битового сдвигового регистра новый бит генерируется с помощью XOR тридцать второго, седьмого, пятого, третьего, второго и первого битов, получающийся ПрСсЛОС будет иметь максимальную длину, циклически проходя до повторения через $2^{32} - 1$ значений.

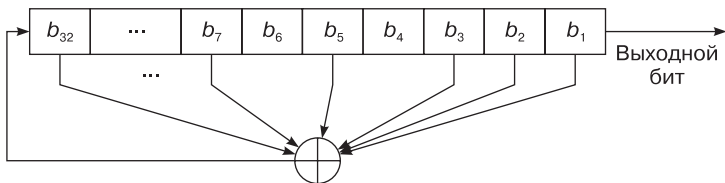


Рис. 3.11. 32-битовый ПрСсЛОС с максимальной длиной

Рассмотрим программный код ПрСсЛОС, у которого отводная последовательность характеризуется многочленом (32, 7, 5, 3, 2, 1, 0). На языке C это выглядит следующим образом:

```
int LFSR ()
{
    static unsigned long ShiftRegister = 1;
    /* Все, кроме 0. */
    ShiftRegister = (((ShiftRegister >> 31)
    ^ (ShiftRegister >> 6)
    ^ (ShiftRegister >> 4)
    ^ (ShiftRegister >> 2)
    ^ (ShiftRegister >> 1)
```

```

    ^ShiftRegister))
    &0x00000001)
    <<31)
    | (ShiftRegister >> 1);
    return ShiftRegister & 0x00000001;
}

```

Если сдвиговый регистр длиннее компьютерного слова, код усложняется, но не намного. В приложении В приведена таблица некоторых примитивных многочленов по модулю 2, будем использовать ее в дальнейшем для выявления некоторых свойств этих многочленов, а также в программной реализации для задания отводной последовательности.

Следует обратить внимание, что у всех элементов таблицы нечетное число коэффициентов. Такая длинная таблица приведена для дальнейшей работы с RrCсЛОС, так как RrCсЛОС часто используются для работы с потоковыми шифрами и в генераторах псевдослучайных чисел. В нашем случае можно использовать многочлены со старшей степенью не более семи.

Если $p(x)$ примитивен, то примитивен и $x^n p(1/x)$, поэтому каждый элемент таблицы на самом деле определяет два примитивных многочлена. Например, если $(a, b, 0)$ примитивен, то примитивен и $(a, a-b, 0)$. Если примитивен $(a, b, c, d, 0)$, то примитивен и $(a, a-d, a-c, a-b, 0)$. Математически:

*если примитивен $x^a + x^b + 1$, то примитивен и $x^a + x^{a-b} + 1$,
 если примитивен $x^a + x^b + x^c + x^d + 1$,
 то примитивен и $x^a + x^{a-d} + x^{a-c} + x^{a-b} + 1$.*

Наиболее просто программно реализуются примитивные трехчлены, так как для генерации нового бита нужно выполнять XOR только двух битов сдвигового регистра (нулевой член не учитывается, т.е. $x^0 = 1$, см. пример выше). Действительно, все многочлены обратной связи, приведенные в таблице, являются разреженными, т.е., у них немного коэффициентов. Разреженность всегда представляет собой источник слабости, которой иногда достаточно для вскрытия алгоритма. Для криптографических алгоритмов гораздо лучше использовать плотные примитивные многочлены, у которых много коэффициентов. Применяя плотные многочлены, особенно в качестве части ключа, можно использовать значительно более короткие RrCсЛОС.

Генерировать плотные примитивные многочлены по модулю 2 не легко. В общем случае для генерации примитивных многочленов степени k нужно знать разложение на множители числа $2^k - 1$.

Сами по себе РгСсЛОС являются хорошими генераторами псевдослучайных последовательностей, но они обладают некоторыми нежелательными неслучайными (детерминированными) свойствами. Последовательные биты линейны, что делает их бесполезными для шифрования. Для РгСсЛОС длины n внутреннее состояние представляет собой предыдущие n выходных битов генератора. Даже если схема обратной связи хранится в секрете, она может быть определена по $2n$ выходным битам генератора с помощью высокоэффективного алгоритма *Berlekamp — Massey*.

Кроме того, большие случайные числа, генерируемые с использованием идущих подряд битов этой последовательности, сильно коррелированы и для некоторых типов приложений вовсе не являются случайными. Несмотря на это, РгСсЛОС часто используются в качестве составных частей систем и алгоритмов шифрования.

О потоковых шифрах на базе РгСсЛОС. Основной подход при проектировании генератора потока ключей на базе РгСсЛОС прост. Сначала берется один или несколько РгСсЛОС обычно с различными длинами и различными многочленами обратной связи. Если длины взаимно просты, а все многочлены обратной связи примитивны, то у образованного генератора будет максимальная длина. Ключ является начальным состоянием регистров РгСсЛОС. Каждый раз для получения нового бита, достаточно сдвинуть на бит регистры РгСсЛОС (это иногда называют тактированием (*clocking*)). Бит выхода представляет собой функцию, желательнo нелинейную, некоторых битов регистров РгСсЛОС. Эта функция называется комбинирующей функцией, а генератор в целом — комбинационным генератором. Если бит выхода является функцией единственного РгСсЛОС, то генератор называется фильтрующим генератором. Большая часть теории подобного рода устройств разработана Селмером (*Selmer*) и Нилом Цирлером (*Neal Zierler*). Можно ввести ряд усложнений. В некоторых генераторах для различных РгСсЛОС используется различная тактовая частота, иногда частота одного генератора зависит от выхода другого. Все это электронные версии идей шифровальных машин, появившихся до Второй мировой войны, которые называются генераторами с управлением тактовой частотой (*clock-controlled generators*). Управление тактовой частотой может быть с прямой связью, когда выход одного РгСсЛОС управляет тактовой частотой другого РгСсЛОС, или с обратной связью, когда выход одного РгСсЛОС управляет его собственной тактовой частотой. Хотя все эти генераторы чувствительны, по крайней мере теоретически, к вскрытиям вложением и вероятной корреляцией, многие из них безопасны до сих пор.

Ян Касселлс (*Ian Cassells*), ранее возглавлявший кафедру чистой математики в Кембридже и работавший криптоаналитиком в Блетчли Парк (*Bletchly Park*), сказал, что «криптография — это смесь математики и путаницы, и без путаницы математика может быть использована против вас». Он имел в виду, что в потоковых шифрах для обеспечения максимальной длины и других свойств необходимы определенные математические структуры, такие как PrCcЛЮС , но, чтобы помешать кому-либо получить содержание регистра и вскрыть алгоритм, необходимо внести некоторый сложный нелинейный беспорядок. Этот совет справедлив и для блочных алгоритмов.

Большинство реальных потоковых шифров основаны на PrCcЛЮС . Даже в первые дни электроники построить их было несложно. Сдвиговый регистр не представляет собой ничего большего, чем массив битов, а последовательность обратной связи — набор вентилей XOR. Даже при использовании современных интегральных схем потоковый шифр на базе PrCcЛЮС обеспечивает немалую безопасность с помощью нескольких логических вентилей. Проблема PrCcЛЮС состоит в том, что их программная реализация очень неэффективна. Вам приходится избегать разреженных многочленов обратной связи — они облегчают корреляционные вскрытия — а плотные многочлены обратной связи неэффективны.

Эта отрасль криптографии быстро развивается и находится под зорким государственным контролем со стороны NSA. Большинство разработок засекречены — множество используемых сегодня военных систем шифрования основаны на PrCcЛЮС . Действительно, у большинства компьютеров *Cray* (*Cray 1*, *Cray X-MP*, *Cray Y-MP*) есть весьма любопытная инструкция, обычно называемая как «счетчик совокупности» (*population count*). Она подсчитывает количество единиц в регистре и может быть использована как для эффективного вычисления расстояния Хэмминга между двумя двоичными словами так и для реализации векторизированной версии PrCcЛЮС . Эта инструкция считается канонической инструкцией NSA, обязательно фигурирующей почти во всех контрактах, касающихся компьютеров.

С другой стороны, было взломано удивительно большое число казавшихся сложными генераторов на базе сдвиговых регистров.

О линейной сложности генерируемой последовательности псевдослучайных чисел PrCcЛЮС . Анализировать потоковые шифры часто проще, чем блочные. Например, важным параметром, используемым для анализа генераторов на базе PrCcЛЮС , является линейная сложность (*linear complexity*), или линейный интервал. Она определяется как длина n самого короткого PrCcЛЮС , который может имитировать выход

генератора. Любая последовательность, сгенерированная конечным автоматом над конечным полем, имеет конечную линейную сложность. Линейная сложность важна, потому что с помощью простого алгоритма, называемого алгоритмом *Berlekamp-Massey*, можно определить этот РгСсЛОС, проверив только $2n$ битов потока ключей. Воссоздавая нужный РгСсЛОС, вы взламываете потоковый шифр.

Эта идея можно расширить с полей на кольца и на случаи, когда выходная последовательность рассматривается как числа в поле нечетной характеристики. Дальнейшее расширение приводит к вводу понятия профиля линейной сложности, который определяет линейную сложность последовательности по мере ее удлинения. Существуют также понятия сферической и квадратичной сложности. В любом случае следует помнить, что высокая линейная сложность не обязательно гарантирует безопасность генератора, но низкая линейная сложность указывает на недостаточную безопасность генератора.

О корреляционной независимости генерируемой последовательности псевдослучайных чисел РгСсЛОС. Криптографы пытаются получить высокую линейную сложность, нелинейно объединяя результаты некоторых выходных последовательностей. При этом опасность состоит в том, что одна или несколько внутренних выходных последовательностей — часто просто выходы отдельных РгСсЛОС — могут быть связаны общим ключевым потоком и вскрыты при помощи линейной алгебры. Часто такое вскрытие называют корреляционным вскрытием, или вскрытием разделяй-и-властвуй. Томас Сигенталер (*Thomas Siegenthaler*) показал, что можно точно определить корреляционную независимость и что существует компромисс между корреляционной независимостью и линейной сложностью.

Основной идеей корреляционного вскрытия является обнаружение некоторой корреляции между выходом генератора и выходом одной из его составных частей. Тогда, наблюдая выходную последовательность, можно получить информацию об этом промежуточном выходе. Используя эту информацию и другие корреляции, можно собирать данные о других промежуточных выходах до тех пор, пока генератор не будет взломан.

Против многих генераторов потоков ключей на базе РгСсЛОС успешно использовались корреляционные вскрытия и их вариации, такие как быстрые корреляционные вскрытия, предлагающие компромисс между вычислительной сложностью и эффективностью.

О других способах вскрытия генерируемой последовательности псевдослучайных чисел РгСсЛОС. Существуют и другие способы вскрытия генераторов потоков ключей. Тест на линейную корректность (*linear*

consistency) — это попытка найти некоторое подмножество ключа шифрования с помощью матричной техники. Существует и вскрытие корректности «встречей посередине» (*meet-in-the-middle consistency attack*). Алгоритм линейного синдрома (*linear syndrome algorithm*) основан на возможности записать фрагмент выходной последовательности в виде линейного уравнения. Существует вскрытие лучшим аффинным приближением (*best affine approximation attack*) и вскрытие выведенным предложением (*derived sequence attack*). К потоковым шифрам можно применить также методы дифференциального и линейного криптоанализа.

На рисунке 3.12 приведена обобщенная схема алгоритма PrCcЛЮС, рассматриваемого в лабораторной работе.

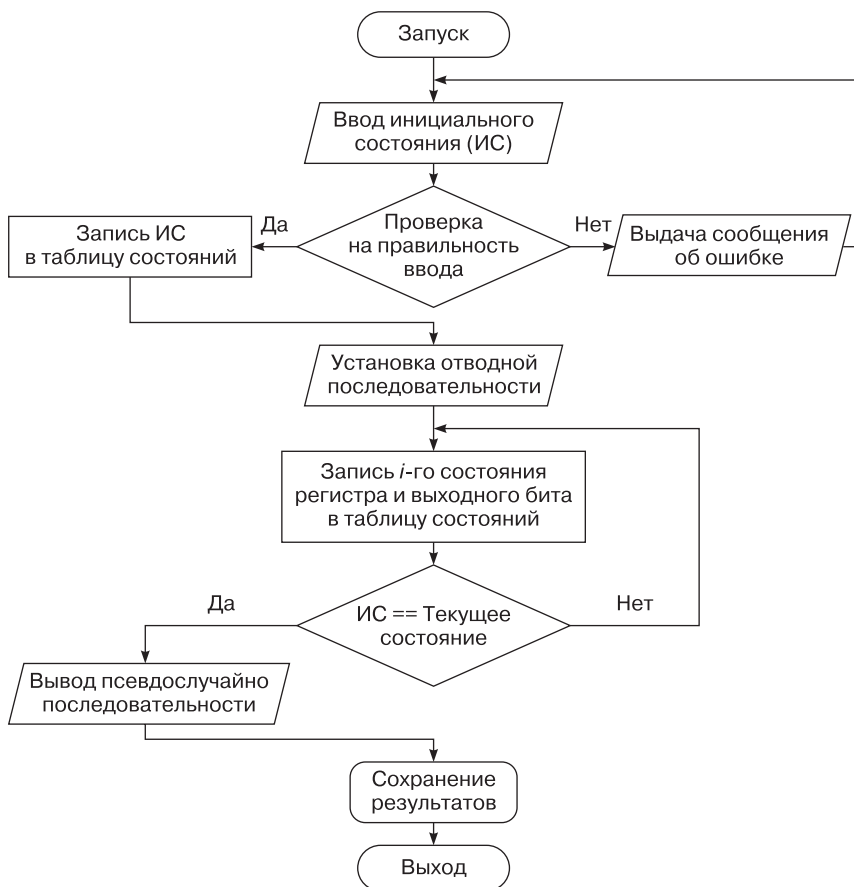


Рис. 3.12. Обобщенная схема алгоритма PrCcЛЮС

Описание программного интерфейса. На рис. 3.13 представлено главное окно программы.

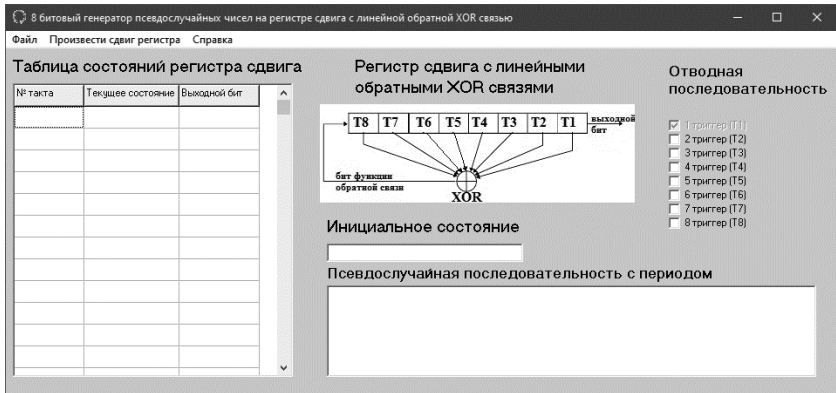


Рис. 3.13. Главное окно программы

В меню есть следующие функции:

■ *Файл* → *Сохранить отчет*

Эта функция осуществляет создание файла отчета, куда записываются инициальное состояние RgCсЛОС, отводная последовательность, период полученной псевдослучайной последовательности бит, сама последовательность и таблица состояний. Если файл успешно был сохранен, то выдается сообщение об успешном сохранении (рис. 3.14). Рекомендуемое расширение файла отчета *.txt. Если сохранение отчета выполняется до внесения значений в поля формы, то выдается сообщение об ошибке (рис. 3.15).

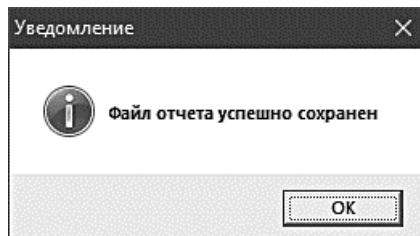


Рис. 3.14. Уведомление об успешном сохранении файла отчета

■ *Файл* → *Выход*

Эта функция обеспечивает закрытие приложения.

■ *Произвести сдвиг регистра*

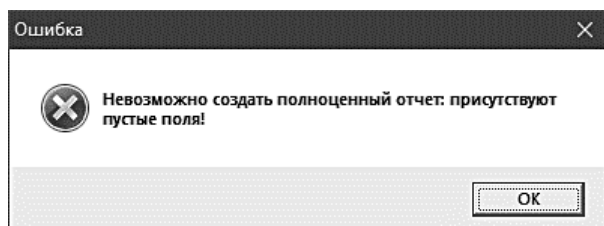


Рис. 3.15. Уведомление об ошибке создания отчета

Эта функция эмулирует работу регистра сдвига. Последовательно производя 256 сдвигов, каждый сдвиг формирует выходной бит псевдослучайной последовательности и новое состояние регистра. Как только находится состояние регистра идентичное инициальному, вычисляется период и выводится в поле «Псевдослучайная последовательность с периодом» полученная псевдослучайная последовательность. В программном инструментарии предусмотрена обработка ошибок ввода значения в поле «Инициальное состояние»:

1. Значение не задано (пустая строка) или задано, но его длина составляет менее 8 бит / 1 байта (рис. 3.16);

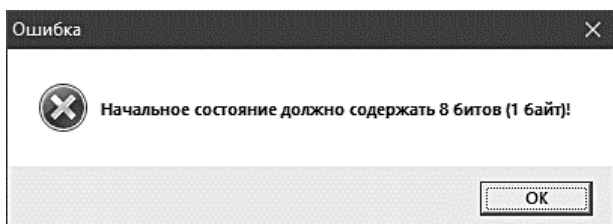


Рис. 3.16. Сообщение о том, что используется недопустимая длина начального состояния

2. Введенное значение состоит не только из нулей и единиц — не является двоичным (рис. 3.17);

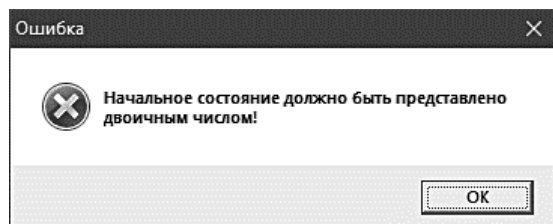


Рис. 3.17. Сообщение о том, что система счисления для указания начального состояния выбрана неверно

3. Введенное значение состоит только из нулей (рис. 3.18).

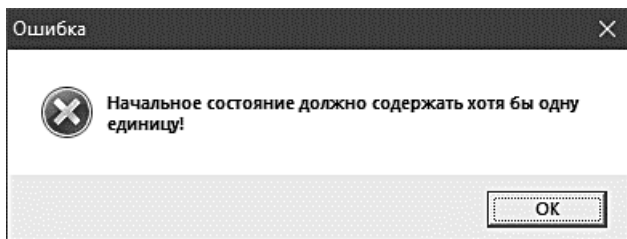


Рис. 3.18. Сообщение о том, что строка начального состояния не содержит единиц

■ *Справка → О программе*

Эта функция выводит на экран краткое описание программы и инструкцию (рис. 3.19).

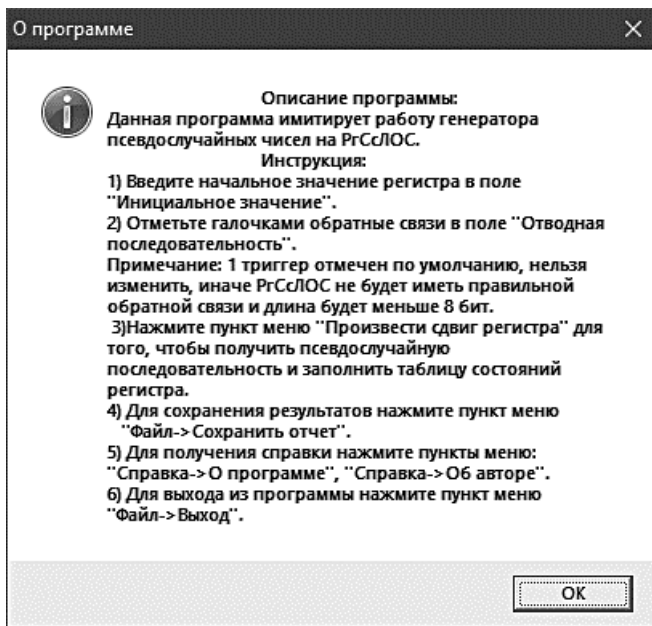


Рис. 3.19. О программе

■ *Справка → Об авторе*

Эта функция выводит на экран информацию об авторе программы (рис. 3.20).

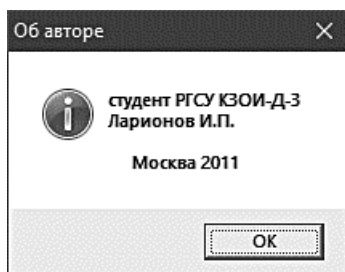


Рис. 3.20. Об авторе

Задание

1. Запустите программу *8bit-LFSR.exe*. Ознакомьтесь со справкой по программе для выполнения работы (см. рис. 3.19).

2. Введите в поле «Инициальное значение» любое восьмиразрядное двоичное число.

3. Установите отводную последовательность (по умолчанию обязательно один триггер должен входить в отводную последовательность и должна присутствовать хотя бы одна обратная связь, иначе регистр будет неисправен).

4. Нажмите пункт меню «Произвести сдвиг регистра».

5. В отчете по лабораторной работе отразите результаты, полученные в п. 4 (**Файл — Сохранить отчет**):

- введенное в п. 2 инициальное значение;
- многочлен, характеризующий отводную последовательность;
- период полученной псевдослучайной последовательности;
- полученную псевдослучайную последовательность.

Ответьте на вопрос: является ли полученная в п. 5 последовательность максимальной? Почему?

6. Выполните п. 3—5 с теми же параметрами, но в отводной последовательности укажите только один триггер.

Ответьте на вопрос: какое значение поступает на вход восьмого триггера при каждом его сдвиге и как оно вычисляется в случае если присутствует одна обратная связь? Если присутствует две или более (до восьми) обратных связей?

7. Выполните п. 3—5 с теми же параметрами, но в качестве инициальной последовательности укажите «11111111».

Ответьте на вопрос: влияет ли инициальное значение на период псевдослучайной последовательности? Влияет ли инициальное значение на псевдослучайную последовательность?

- Измените отводную последовательность (должно быть минимум две связи), а инициальное значение оставьте тем же. Произведите сдвиг регистра. Что изменилось по сравнению с предыдущим пунктом?
- Измените отводную последовательность, установив только одну обратную связь T1. Произведите сдвиг регистра. Что изменилось по сравнению с предыдущим пунктом? Почему период последовательности отличается от аналогичных в предыдущих пунктах (п. 6—7)?

8. Выполните п. 3—5 с теми же параметрами, но в качестве инициальной последовательности укажите «00000000».

Ответьте на вопрос: почему это значение недопустимо при работе PтСсЛОС?

9. Выполните п. 3—5, но в отводной последовательности отметьте галочками 1, 5, 6 и 7 триггеры или 1, 4, 6, 8 триггеры.

- Измените несколько раз инициальное значение и произведите сдвиг регистра.

Ответьте на вопросы: Каков период псевдослучайной последовательности? Почему он принимает это значение? Какой многочлен по модулю два задает такую последовательность? Является ли он приводимым? Что меняется при изменении инициального значения?

10. Предоставьте электронный отчет преподавателю. Отчет должен содержать ответы на вопросы п. 5—10, сведения для п. 5—10, описанные в п. 5, 1, и скриншот главной формы и формы «Справка — О программе» (п. 1).

11. Включите в отчет о лабораторной работе ответы на вопросы, выбранные в соответствии с номером варианта из табл. 3.5.

Таблица 3.5

Номер варианта	Контрольные вопросы
1, 5, 7, 3, 9, 18, 28	Что такое М-последовательность? Каковы ее свойства? Какая отводная последовательность позволяет ее получить? Опишите процесс работы четырехбитового РгСсЛОС: откуда берется выходной бит и как формируется псевдослучайная последовательность, как происходит сдвиг регистра, как меняется его состояние, как образуется бит функции обратной связи (приведите таблицу истинности для операции XOR от 1, 2, 3, 4 переменных)
2, 4, 6, 8, 20, 22, 24, 26, 30	Что определяет свойство периодичности РгСсЛОС? Отчего зависит период РгСсЛОС? Какие многочлены являются неприводимыми по модулю 2? Где и для чего их можно применять на практике?
11, 13, 15, 10, 17, 19, 27	Что входит в понятие «Линейная сложность бинарной последовательности»? Как ее можно использовать для оценки псевдослучайно бинарной последовательности? Что определяет свойство периодичности РгСсЛОС? Отчего зависит период РгСсЛОС?
12, 14, 16, 21, 23, 25, 29	Что такое отводная последовательность? Для чего она нужна в РгСсЛОС и на какие его параметры влияет? Что такое инициальное состояние РгСсЛОС? Какие есть ограничения на значение инициального состояния? Почему? Что такое ГСПЧ? На чем они могут быть основаны? Какие у них преимущества и недостатки?

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Задача защиты информации от несанкционированного доступа решалась самыми разнообразными способами на различных этапах истории. Уже в древнем мире выделилось два основных направления решения этой задачи, существующие и по сегодняшний день: *криптография* и *стеганография*. Целью криптографии, как уже указывалось, является скрытие содержимого сообщений за счет их шифрования. В отличие от этого стеганография скрывает сам факт существования тайного сообщения.

Слово «*стеганография*» в переводе с греческого буквально означает «тайнопись» (*steganos* — секрет, тайна; *graphy* — запись). Тайнопись осуществляется самыми различными способами. Общей чертой этих способов является то, что скрываемое сообщение встраивается в некоторый безобидный, не привлекающий внимание объект. Затем этот объект открыто передается адресату, не вызывая какого-либо подозрения со стороны.

В различные времена и эпохи люди применяли самые разнообразные стеганографические методы для защиты своих секретов. Местом зарождения стеганографии многие называют Египет, хотя первыми «стеганографическими сообщениями» можно назвать и наскальные рисунки древних людей.

Первое упоминание о стеганографических методах в литературе приписывается Геродоту, описавшему случай передачи сообщения Демартом, который хотел послать в Спарту сообщение об угрозе нападения Ксерксов. Тогда он соскоблил воск с дощечки, написал послание непосредственно на дереве, затем вновь покрыл ее воском. В результате доска выглядела неиспользованной и без проблем прошла досмотр центурионов.

Еще один, весьма неожиданный способ сокрытия информации или условных знаков — татуировка на голове бритого посланца. Для передачи тайного сообщения голову раба обривали, наносили на кожу татуировку, и когда волосы отрастали и сообщение становилось неви-

димым, его отправляли с посланием. Для прочтения требовалось побрить голову гонца.

В Китае письма писали на полосках шелка. Поэтому для сокрытия сообщений полоски с текстом письма сворачивались в шарики, покрывались воском и затем глотались посыльными.

Темное средневековье породило не только инквизицию: усиление слежки привело к развитию как криптографии, так и стеганографии. В XV веке монах Тритемиус (1462—1516), занимавшийся криптографией и стеганографией, описал множество различных методов скрытой передачи сообщений. Позднее, в 1499 г., эти записи были объединены в книгу *Steganographia*.

К стеганографии также относится написание текстов невидимыми (симпатическими) чернилами, проявляющимися при нагревании. Яркий пример применения таких чернил — история с В. И. Лениным, который в тюрьме писал статьи в «Искру» молоком между строк книги.

Особое место в истории стеганографии занимают фотографические микроточки, которые сводили с ума спецслужбы США во время Второй мировой войны. Однако микроточки появились намного раньше, сразу же после изобретения Луи Жак Манде Дагером фотографического процесса, и впервые в военном деле были использованы во времена франко-прусской войны в 1870 г.

Распространение стеганографии во время Второй мировой войны и тотальная шпиономания вызвали появление многих цензурных ограничений. В США были запрещены к международной почтовой пересылке описания шахматных партий, инструкции по вязанию и шитью, вырезки из газет, детские рисунки. Запрещалось посылать телеграммы с указанием доставить определенный сорт цветов к определенной дате, а впоследствии американским и английским правительствами были запрещены вообще все международные телеграммы, касающиеся доставки и заказа цветов.

В настоящее время в связи с бурным развитием вычислительной техники и новых каналов передачи информации появились и новые стеганографические методы, в основе которых лежат особенности представления информации в компьютерных файлах и вычислительных сетях. Это дает возможность говорить о становлении нового направления — цифровой или компьютерной стеганографии.

Как и любой новый раздел науки, компьютерная стеганография, несмотря на большое количество открытых публикаций и ежегодные конференции, долгое время не имела единой терминологии. Только на конференции *Information Hiding: First Information Workshop*

в 1996 г. было предложено использовать единую стеганографическую терминологию.

Стеганографическая система или *стегосистема* — совокупность средств и методов, которые используются для формирования скрытого канала передачи информации. При построении стегосистемы должны учитываться следующие положения:

- противник имеет полное представление о стеганографической системе и деталях ее реализации, единственной информацией, которая остается неизвестной потенциальному противнику, является ключ, с помощью которого только его держатель может установить факт присутствия и содержание скрытого сообщения;
- если противник каким-то образом узнает о факте существования скрытого сообщения, это не должно позволить ему извлекать подобные сообщения из других объектов до тех пор, пока ключ хранится в тайне;
- потенциальный противник должен быть лишен каких-либо технических и иных преимуществ в распознавании или раскрытии содержания тайных сообщений.

В качестве встраиваемых данных может использоваться любая информация: текст, звук, графика и пр.

Контейнер — любая информация, предназначенная для сокрытия тайных сообщений. Под *пустым контейнером* понимается контейнер без встроенного сообщения; *заполненный контейнер* или *стегоконтейнер* содержит встроенную информацию.

Встроенное (скрытое) сообщение — это сообщение, встраиваемое в контейнер.

Стеганографический канал, или *стегоканал* — канал передачи стего.

Стегоключ, или *ключ* — секретный ключ, необходимый для сокрытия информации. В зависимости от количества уровней защиты (например, встраивание предварительно зашифрованного сообщения) в стегосистеме может быть один или несколько стегоключей.

По типу ключа стегосистемы аналогично криптосистемам можно подразделить на два типа: *системы с секретным* и *с открытым ключом*.

В стегосистеме с секретным ключом используется один ключ, который должен быть определен либо до начала обмена секретными сообщениями, либо передан по защищенному каналу.

В стегосистеме с открытым ключом для встраивания и извлечения сообщения используются разные ключи, которые различаются таким образом, что с помощью вычислений невозможно вывести секретный

ключ из открытого. Поэтому открытый ключ может передаваться свободно по незащищенному каналу связи. Такая схема хорошо работает и при взаимном недоверии отправителя и получателя.

Любая стegosистема должна отвечать следующим требованиям:

- свойства контейнера должны быть таковы, чтобы изменение невозможно было выявить при визуальном контроле; это требование определяет качество сокрытия внедряемого сообщения: для обеспечения беспрепятственного прохождения стегосообщения по каналу связи оно никоим образом не должно привлечь внимание атакующего;
- стегосообщение должно быть устойчиво к искажениям, в том числе и злонамеренным; в процессе передачи изображение (звук или другой контейнер) может претерпевать различные трансформации: масштабироваться, преобразовываться в другой формат, кроме того, оно может быть сжато, в том числе и с использованием алгоритмов сжатия с потерей данных;
- для сохранения целостности встраиваемого сообщения необходимо использование, например, кодов с обнаружением и исправлением ошибок;
- для повышения надежности встраиваемое сообщение должно быть продублировано.

В настоящее время можно выделить три тесно связанных между собой и имеющих одни корни направления стеганографии: *сокрытие данных, цифровые водяные знаки и заголовки*.

Сокрытие внедряемых данных — это основное направление компьютерной стеганографии. На его основе базируются все остальные разделы этой развивающейся науки. Если говорить о сокрытии информации для ее секретной передачи, то в данном случае встраиваемое сообщение обычно имеет большой размер и размер контейнера в несколько раз должен превышать объем встраиваемых данных.

Цифровые водяные знаки (ЦВЗ) используются для защиты авторских или имущественных прав на цифровые изображения, фотографии или другие оцифрованные произведения искусства. Основными требованиями, которые предъявляются к таким встроенным данным, являются надежность и устойчивость к искажениям. ЦВЗ имеют небольшой объем, однако с учетом указанных выше требований для их встраивания используются более сложные методы, чем для встраивания просто сообщений или заголовков.

Заголовки используются в основном для маркирования изображений в больших электронных хранилищах (библиотеках) цифровых

изображений, аудио и видео файлов. Внедряемые заголовки имеют небольшой объем, а предъявляемые к ним требования минимальны: заголовки должны вносить незначительные искажения и быть устойчивы к основным геометрическим преобразованиям.

Среди основных причин наблюдающегося всплеска интереса к стеганографии можно выделить принятые в ряде стран ограничения на использование сильной криптографии, а также проблему защиты авторских прав на художественные произведения в цифровых глобальных сетях. Поэтому в ближайшее время можно ожидать совершенствования существующих и разработки новых методов и алгоритмов сокрытия передаваемой информации.

ЛАБОРАТОРНАЯ РАБОТА № 13

ЗАЩИТА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ МЕТОДАМИ СТЕГАНОГРАФИИ

Цель работы: ознакомление с методами защиты исполняемых файлов. Изучение возможностей стеганографической защиты .exe файлов путем встраивания цифровых водяных знаков в пустое место, в конце секции файла.

Примечание. Для выполнения лабораторной работы на компьютере необходимо установить программу *Filigrana.exe*

Описание лабораторной работы. *Защита программного обеспечения (ПО)* — комплекс мер, направленных на защиту ПО от несанкционированного приобретения, использования, распространения, модифицирования, изучения и воссоздания аналогов.

Разработка наиболее эффективного метода защиты для того или иного программного продукта в настоящее время становится одной из важных задач большинства программистов, которые занимаются разработкой специализированного, платного программного обеспечения, так как это позволяет им продавать свой интеллектуальный труд и исключить возможности его нелегального использования среди потребителей, говоря иными словами, никто не сможет использовать оригинальную, лицензионную копию определенной программы, предварительно не купив, не заплатив денег ее разработчику.

Затраты производителей на создание эффективного метода защиты их программных продуктов окупаются и компенсируют потенциальный ущерб, наносимый нелегальным копированием и использованием программ.

Существуют два основных способа защиты интеллектуальной собственности и, следовательно, самих программных продуктов:

1) *юридический* — данный способ защиты заключается в создании определенных законодательных актов, которые будут охранять интеллектуальную собственность (в нашем случае программные продукты) от нелегального использования;

2) *технический* — реализуется путем включения в программный продукт какого-либо из существующих методов защиты, который будет запрещать его нелегальное использование. По сравнению с юридическим способом защиты он является наиболее распространенным, так как он практичен и сравнительно недорогой в реализации.

Юридическая защита. Юридическая защита включает в себя такие методы как патентование, оформление авторских прав на интеллектуальную собственность и т.д. Также предусматривается возможность лицензирования программного продукта, так, например большинство программных продуктов поставляются вместе с лицензией, которая подтверждает право пользователя использовать этот программный продукт, т.е., покупая лицензионную копию программы, пользователь в некой мере производит покупку лицензии на право работы с ее копией.

Техническая защита. Существует множество технических методов защиты программных продуктов. Ниже будут рассмотрены наиболее интересные и актуальные из них.

Использование парольной защиты (кода активации). Этот вариант на сегодня является самым распространенным. Принцип действия защиты основан на идентификации и аутентификации пользователя ПО путем запроса у него дополнительных данных, это могут быть название фирмы и (или) имя и фамилия пользователя и его пароля либо только пароля. Эта информация может запрашиваться в различных ситуациях, например при старте программы, по истечении срока бесплатного использования ПО, при вызове процедуры регистрации или в процессе установки на компьютер пользователя. Процедура защиты проста в реализации и поэтому очень часто применяется производителями ПО. Она сводится к проверке правильности вводимого кода и запуске или незапуске ПО в зависимости от результатов проверки.

Защита через Интернет. Данный метод защиты схож с предыдущим, его отличие заключается в том, что проверка кода активации происходит на сервере производителя. Если ваша программа работает с сетью (это может быть менеджер закачки файлов, браузер, FTP-клиент и пр.), можно применить проверку введенного в программу кода через Интернет, используя базу данных пользователей на сайте программы. Однако проверять нужно какие-либо косвенные данные, чтобы исследователь не мог получить с сайта программы, например, базу данных с серийными номерами.

Сложность взлома этой защиты в том, что весьма непросто найти в коде программы место, где идет обращение к Интернет-серверу именно с целью проверки регистрационного кода.

Выполнение программ на стороне сервера. Данный метод защиты основан на технологии клиент-сервер, он позволяет предотвратить отсылку кода программы пользователям, которые будут с ней работать, так как сама программа хранится и выполняется на сервере, а пользователи, используя клиентскую часть этой программы, получают результаты ее выполнения (рис. 4.1).

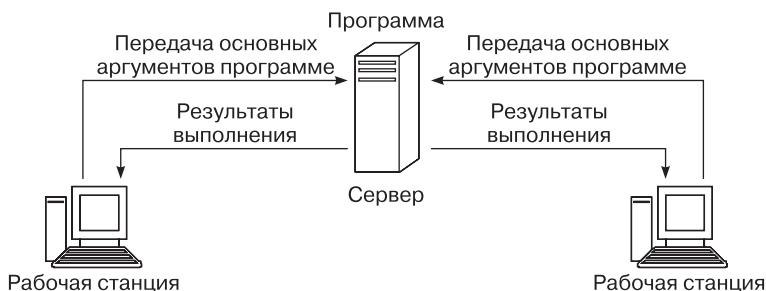


Рис. 4.1. Выполнение программы на стороне сервера

Клиентскую часть можно распространять бесплатно, это позволит пользователям платить только за использование серверной части.

Недостатком данного метода является то, что он устанавливает зависимость пропускной способности сети и тех данных, с которыми будет работать программа (соответственно работа с мультимедиа данными требует максимальной пропускной способности сети). Поэтому данный метод наиболее эффективен для простых программ (сценариев), потребность в которых очень велика среди пользователей.

Преимущество такого метода заключается в том, что злоумышленнику в данном случае нужно будет для начала скомпрометировать сам сервер, только после чего он сможет получить копию требуемой программы.

Данный метод не позволяет защитить программу от нелегального копирования, поэтому после того как ее копия попадет к злоумышленнику, он сможет делать с ней что захочет.

Установка подлинности кода (tamper-proofing). В данном случае в программу помещается процедура проверки целостности самой программы, что позволяет определить, была ли программа изменена (были ли внесены какие-либо изменения в ее код). Если эта процедура обнаруживает, что в программу внесены изменения, она делает программу нефункциональной (рис. 4.2).

Это позволяет защитить программный продукт от модификации со стороны злоумышленника.

Существуют такие пути проверки целостности программы, как:

- проверка идентичности оригинальной и запускаемой программы. Обычно для этого определяется контрольная сумма запущенной программы, которая потом сверяется с записанной в процедуру проверки контрольной суммой оригинальной программы; для осуществления быстрой проверки используют такие алгоритмы как CRC или MD4/5;

- проверка результатов работы программы, т.е. осуществляется проверка выходных значений функций, которые очень чувствительны, к каким либо возможным изменениям в программном коде;
- создание запускаемой программы на лету в соответствии с ее оригинальным образом — это позволяет избежать возможности выполнения изменений внесенных в программу, так как они не будут учитываться при ее создании.

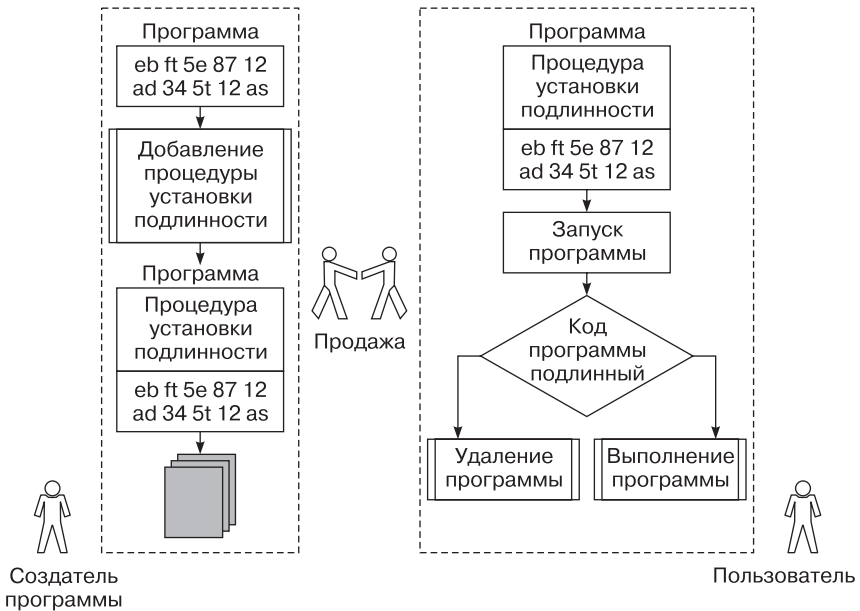


Рис. 4.2. Установка подлинности кода

Шифрование программного кода. Используется для того, чтобы предотвратить вмешательство в программу, а также усложнить процесс понимания взломщиком того, как устроена программа, как она работает, как в ней реализован метод защиты и т.д.

Данный метод защиты предусматривает шифрование кода программы, после чего она в зашифрованном виде поставляется конечным пользователям (иногда эффективно зашифровывать только наиболее важные, критические, участки кода, а не весь код программы). Когда пользователь запускает такую программу, вначале будет запущена процедура расшифровки программы, которой потребуется ключ, с помощью которого и будет расшифрована запускаемая программа (рис. 4.3).

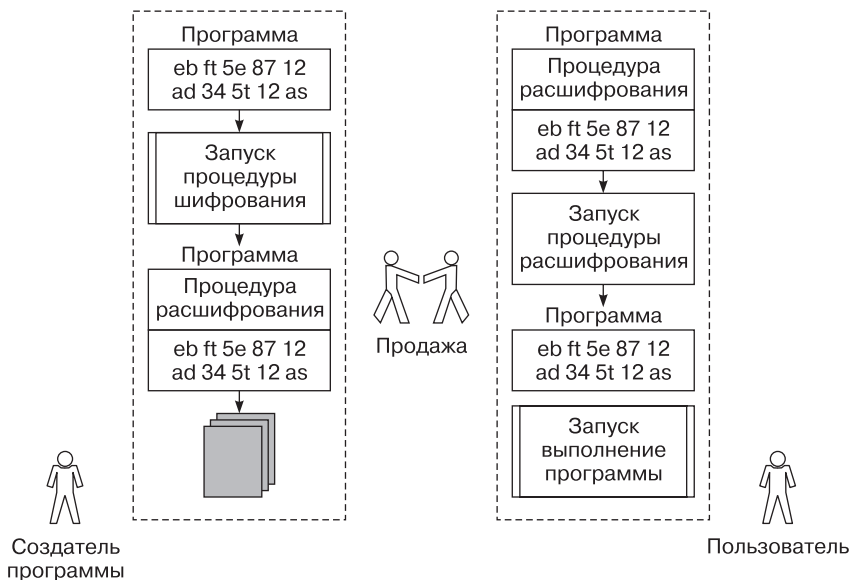


Рис. 4.3. Шифрование программного кода

Способ шифрования программ имеет недостатки, одним из которых является то, что у взломщика есть возможность после приобретения лицензионной копии программы произвести извлечение расшифрованных частей программы в процессе ее работы из памяти. Поэтому сразу после исполнения расшифрованного кода его необходимо выгружать из памяти.

Обфускация программного кода. Обфускация (*obfuscation* — запутывание), это один из методов защиты программного кода, который позволяет усложнить процесс реверсивной инженерии кода защищаемого программного продукта.

Суть процесса обфускации заключается в том, чтобы запутать программный код и устранить большинство логических связей в нем, т.е. трансформировать его так, чтобы он был очень труден для изучения и модификации посторонними лицами (будь то взломщики или программисты которые собираются узнать уникальный алгоритм работы защищаемой программы).

Использование только обфускации для обеспечения полной и эффективной защиты программных продуктов недостаточно, так как она не предоставляет возможности предотвращения нелегального использования программного продукта. Поэтому обфускацию обычно

используют вместе с одним из существующих методов защиты (например, шифрование программного кода), это позволяет значительно повысить уровень защиты программного обеспечения в целом (рис. 4.4).

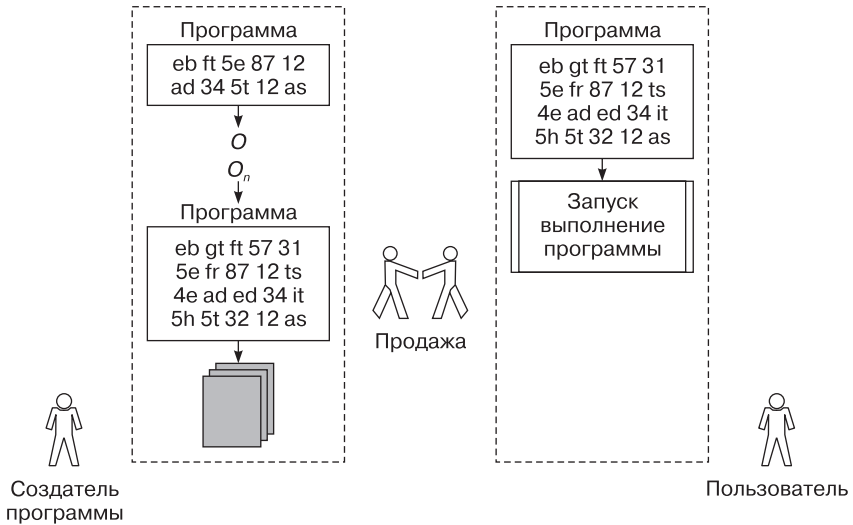


Рис. 4.4. Обфускация программного кода

Так как код, получаемый после осуществления обфускации над одной и той же программой, разный, то процесс обфускации можно использовать для быстрой локализации нарушителей авторских прав (т.е. тех покупателей, которые будут заниматься нелегальным распространением купленных копий программ). Для этого определяют контрольную сумму каждой копии программы, прошедшей обфускацию, и записывают ее вместе с информацией о покупателе в соответствующую базу данных. После этого для определения нарушителя достаточно будет, определив контрольную сумму нелегальной копии программы, сопоставить ее с информацией, хранящейся в базе данных.

Программы-протекторы. Протекторами называются программы, предназначенные для защиты программ от взлома. Данный способ защиты программы стал весьма популярным в последнее время. После того как получен работающий исполняемый файл, этот файл обрабатывается с помощью программы-протектора и создается новый исполняемый файл, в котором реализованы различные средства защиты

(защита программ представляет собой обычную упаковку всего файла, при этом «защищается» сам распаковщик, который в итоге распаковывает файл). Протекторы пишутся профессионалами в области защиты и обеспечивают неплохой уровень противодействия взлому, с которым большинству исследователей программ не справиться.

Недостатком данного метода является то, что протекторы становятся очень популярными и, соответственно, активно изучаются взломщиками, которые пишут программы-антипротекторы (распаковщики защиты). Подобные средства быстро и автоматически снимают защиту.

Использование электронных ключей. На сегодня это наиболее надежный и удобный метод защиты тиражируемого ПО средней и высшей ценовой категории. Он обладает высокой стойкостью к взлому и не ограничивает использование легальной копии программы.

Электронный ключ представляет собой небольшое устройство, которое подсоединяется к одному из портов компьютера (COM, LPT, USB). Принцип его действия состоит в следующем: ключ присоединяется к определенному интерфейсу компьютера, далее защищенная программа через специальный драйвер отправляет ему запрос, который обрабатывается в соответствии с заданным алгоритмом и возвращается обратно. Если ответ ключа правильный, то программа продолжает свою работу. В противном случае она может выполнять любые действия, заданные разработчиками (например, переключаться в демонстрационный режим, блокируя доступ к определенным функциям). Вот некоторые характерные запросы:

- проверка наличия подключения ключа;
- считывание с ключа необходимых программе данных в качестве параметра запуска;
- запрос на расшифрование данных или исполняемого кода, необходимых для работы программы (предварительно разработчик защиты шифрует часть кода программы, и при этом непосредственное выполнение такого зашифрованного кода приводит к ошибке);
- проверка целостности исполняемого кода путем сравнения его текущей контрольной суммы с оригинальной контрольной суммой, считываемой с ключа;
- запрос к встроенным в ключ часам реального времени (при их наличии) и т.д.

Специфика PE-файлов. Типичный исполняемый PE-файл имеет следующую структуру (рис. 4.5).

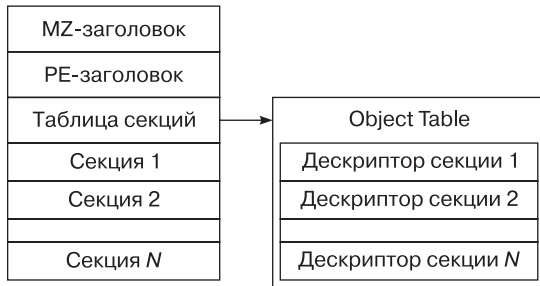


Рис. 4.5. Обобщенная структура PE-файла

MZ-заголовок содержит программу для ОС DOS — эта программа называется stub и нужна для совместимости со старыми ОС. Если мы запускаем PE-файл под ОС DOS или OS/2, она выводит на экран консоли текстовую строку, которая информирует пользователя, что данная программа не совместима с данной версией ОС: *This program cannot be run in DOS mode*. После этой DOS-программы идет структура, которая называется IMAGE_NT_HEADERS (PE-заголовок). Она описывается следующим образом:

```
typedef struct _IMAGE_NT_HEADERS
{
    DWORD Signature;
    IMAGE_FILE_HEADER FileHeader;
    IMAGE_OPTIONAL_HEADER32 OptionalHeader;
}
```

Первый элемент IMAGE_NT_HEADERS — сигнатура PE-файла, которая равна 0x00004550 или в ASCII-символах, «PE00». Кстати, именно из-за этой сигнатуры исполняемый файл Windows и называют PE-файлом.

Далее идет структура, которая называется файловым заголовком и определена как IMAGE_FILE_HEADER. Файловый заголовок содержит наиболее общие свойства для данного PE-файла — тип процессора, для которого скомпилирован исполняемый файл; количество заголовков секций; размер опционального заголовка, а также некоторую другую информацию.

После файлового заголовка идет опциональный заголовок — IMAGE_OPTIONAL_HEADER32. Он содержит специфические

параметры данного PE-файла — размер исполняемого кода; относительный виртуальный адрес (RVA — Relative Virtual Address) точки входа в программу; базовый адрес, начиная с которого в память будет отображен образ исполняемого файла и многое другое. Также здесь хранится немаловажная информация о границах выравнивания секций в памяти — *SectionAlignment* и границах выравнивания секций в файле на диске — *FileAlignment*. Наличие двух видов выравнивания обусловлено спецификой работы компьютера — чтение информации с жесткого диска производится секторами, а из памяти — страницами. То есть если файл занимает 1 байт, с диска будет все равно прочитано 512 (один сектор). Поэтому размер файла на диске дополняется нулями (выравнивается) до определенного значения (обычно 512). То же самое в памяти, только там выравнивание обычно происходит на одну страницу (1000h). Проще говоря, выравнивание — это округление до определенной константы, нужное для оптимизации работы компьютера.

Прежде чем перейти к дальнейшему описанию, нужно прояснить одну важную вещь — PE-файл состоит из секций. Одна секция может содержать код, данные, таблицу импорта или что-либо еще. Главная сложность заключается в том, что секция на диске не обязательно соответствует секции в памяти. Это происходит по нескольким причинам: отчасти из-за выравнивания, о котором было сказано выше, а еще потому, что существует такая вещь, как неинициализированные данные. К примеру, если в листинге написать: `buffer rb 1000h`, то размер файла на диске не увеличится ни на байт. Другое дело в памяти, здесь он будет больше ровно на 1000h. Информация о секциях хранится в специальной таблице (Object Table). Она идет сразу же после PE-заголовка и состоит из набора заголовков секций. Каждый заголовок описывает одну секцию файла: ее размер, размещение на диске и в памяти, параметры и т.д. Все заголовки секции имеют длину 40 байт и располагаются без выравнивания.

За всеми заголовками в файле следуют тела секций. В Microsoft исторически сложились такие устойчивые названия секций: `.text`, `.bss`, `.data`, `.rdata`, `.edata`, `.idata`, `.rsrc`, `.debug` и некоторые другие. Однако совсем не обязательно, чтобы секции имели такие названия. Например, компиляторы от фирмы Inprise (бывшая Borland) присваивают секциям имена вида `CODE`, `DATA` и т.п. Кроме того, программист может создавать дополнительные секции со своими названиями. Наличие точки в начале секции также необязательно. Секции расположены вплотную друг к другу без промежутков, но это совсем не значит, что в самих сек-

циях не бывает пустых мест. Каждая стандартная секция имеет определенное назначение, например:

- `v.text` расположен код программы;
- `v.bss` размещаются неинициализированные данные;
- `v.data` — инициализированные данные;
- `v.edata` — функции, экспортируемые файлом;
- `v.idata` — таблицы импортируемых функций;
- `v.rsrc` — ресурсы;
- в сегменте `.ldata` — данные только для чтения (строки, константы, информация отладочного каталога).

Но это назначение секций не строгое, например `.text` вполне может быть начинен обычными данными, а также экспортируемыми функциями. Некоторые секции имеют особый формат.

В самом конце PE-файла за секциями вполне могут размещаться дополнительные данные, например какая-нибудь отладочная информация, но это носит необязательный характер.

Техника внедрения кода в PE-файлы. Существуют следующие способы внедрения кода в PE-файлы:

- а) размещение кода поверх оригинальной программы (также называемое затиранием);
- б) размещение кода в свободном месте программы (интеграция);
- в) дописывание кода в начало, середину или конец файла с сохранением оригинального содержимого;
- г) размещение кода вне основного тела файла-носителя (например, в динамической библиотеке).

Поскольку, способ а) приводит к необратимой потере работоспособности исходной программы и реально применяется только в вирусах, здесь он не рассматривается. Все остальные алгоритмы внедрения полностью или частично обратимы.



Рис. 4.6. Условные графические обозначения, принятые в дальнейшем изложении

Классификация механизмов внедрения. Механизмы внедрения можно классифицировать по-разному: по месту (начало, конец, середи-

на), «геополитике» (затирание исходных данных, внедрение в свободное пространство, переселение исходных данных на новое место обитания), надежности (предельно корректное, вполне корректное и крайне некорректное внедрение), рентабельности (рентабельное или нерентабельное) и т.д. Следующая классификация основана на характере воздействия на физический и виртуальный образ программы. Все существующие механизмы внедрения можно разделить на четыре категории.

К *категории А* относятся механизмы, не вызывающие изменения адресации ни физического, ни виртуального образов. После внедрения в файл ни его длина, ни количество выделенной при загрузке памяти не изменяется и все базовые структуры остаются на своих прежних адресах. Этому условию удовлетворяют: внедрение в пустое место файла (РЕ-заголовок, хвосты секций, регулярные последовательности), а также внедрение путем сжатия части секции.

К *категории В* относятся механизмы, вызывающие изменения адресации только физического образа. После внедрения в файл его длина увеличивается, однако количество выделенной при загрузке памяти не изменяется и все базовые структуры проецируются по тем же самым адресам, однако их физические смещения изменяются, что требует полной или частичной перестройки структур, привязывающихся к своим физическим адресам, и если хотя бы одна из них останется не скорректированной (или будет скорректирована неправильно), файл-носитель с высокой степенью вероятности откажет в работе. Категории В соответствуют: раздвижка заголовка, сброс части оригинального файла в оверлей и создание своего собственного оверлея.

К *категории С* относятся механизмы, вызывающие изменения адресации как физического, так и виртуального образов. Длина файла и выделяемая при загрузке память увеличиваются. Базовые структуры могут либо оставаться на своих местах (т.е. изменяются лишь смещения, отсчитываемые от конца образа (файла)), либо перемещаться по страничному имиджу произвольным образом, требуя обязательной коррекции. Этой категории соответствует: расширение последней секции файла, создание своей собственной секции и расширение серединных секций.

К «засекреченной» *категории Z* относятся механизмы, вообще не затрагивающие файла-носителя и внедряющиеся в его адресное пространство косвенным путем, например модификацией ключа реестра, ответственного за автоматическую загрузку динамических библиотек. Эту технологию в первую очередь используют сетевые черви и шпионы.

Категория А: внедрение в пустое место файла. Проще всего внедриться в пустое место файла. На сегодня таких мест известно три:

- 1) PE-заголовок;
- 2) хвостовые части секций;
- 3) регулярные последовательности.

Рассмотрим их подробнее.

Внедрение в PE-заголовок

Типичный PE-заголовок вместе с MS-DOS заголовком занимает порядка $\sim 300h$ байт, а минимальная кратность выравнивания секций составляет $200h$ байт. Таким образом, между концом заголовка и началом первой секции практически всегда имеется $\sim 100h$ байт, которые можно использовать для внедрения скрытого сообщения, размещая здесь либо всю внедряемую информацию целиком, либо ее часть, если размер не позволяет большего (рис. 4.7).

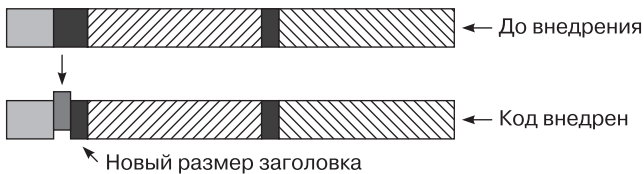


Рис. 4.7. Внедрение информации в свободное пространство хвоста PE-заголовка

Перед внедрением в заголовок необходимо убедиться, что хвостовая часть заголовка действительно свободна. Сканирование таких заголовков обычно выявляет длинную цепочку нулей, расположенных в его хвосте и, очевидно, никак и никем не используемых. Туда-то и можно записать информацию, но только с предосторожностями. Необходимо отсчитать по меньшей мере $10h$ байт от последнего ненулевого символа, оставляя этот участок нетронутым (в конце некоторых структур присутствует до $10h$ нулей, искажение которых ни к чему хорошему не приведет).

Внедрение в PE-заголовок в большинстве случаев можно распознать визуально. Рассмотрим, как выглядит в hex-редакторе типичный исполняемый файл `notepad.exe` (рис. 4.8): как уже было сказано выше, вслед за концом MS-DOS заголовка, обычно содержащим в себе строку `This program cannot be run in DOS mode`, расположена «PE» сигнатура, за которой следует немного мусора, разбавленного нулями и плавно перетекающего в таблицу секций, содержащую легко узнаваемые имена `.text`, `.rsrc` и `.data`.

Внедрение в хвост секции

Операционная система Windows требует, чтобы физические адреса секций были выровнены по меньшей мере на 200h байт, поэтому между секциями практически всегда есть некоторое количество свободного пространства, в которое легко можно внедрить информацию (рис. 4.10).

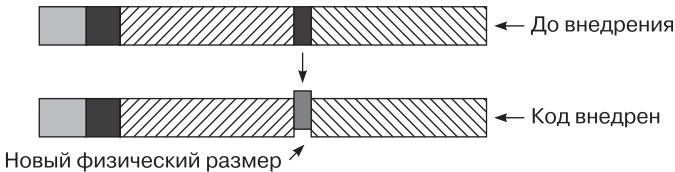


Рис. 4.10. Внедрение кода в хвост секции, оставшийся от выравнивания

Перед внедрением необходимо найти секцию с подходящими атрибутами и достаточным свободным пространством в конце или сосредоточить внедряемый код в нескольких секциях. Для этого необходимо просканировать хвостовую часть секции на предмет наличия непрерывной цепочки нулей — если таковая там действительно присутствует, ее можно безбоязненно использовать для внедрения.

Распознать внедрение этого типа достаточно трудно, особенно если код полностью помещается в первой кодовой секции файла, которой, как правило, является секция .text. Внедрение в секцию данных разоблачает себя наличием характерного кода в ее хвосте.

Внедрение осмысленного машинного кода в хвост секции данных представлено на рис. 4.11.



Рис 4.11. Осмысленный машинный код в хвосте секции данных

Внедрение в регулярную последовательность байт

Цепочки нулей необязательно искать в хвостах секций, также как необязательно искать именно нули, для внедрения подходит любая регулярная последовательность (например, цепочка FF FF FF.. или даже FF 00 FF 00...). Если внедряемых цепочек больше одной, код придется разбить по всему телу файла. Соответственно стартовые адреса и длины этих цепочек придется где-то хранить для их последующего восстановления.

Регулярные последовательности чаще всего обнаруживаются в ресурсах, а точнее — в bitmap'ах и иконках. Технически внедрить сюда ничего не стоит, но пользователь тут же заметит искажение иконки, чего допускать ни в коем случае нельзя (даже если это и не главная иконка приложения, так как проводник показывает остальные по нажатию кнопки «сменить значок» в меню свойств ярлыка). Внедрение кода в регулярные цепочки продемонстрировано на рис. 4.12.

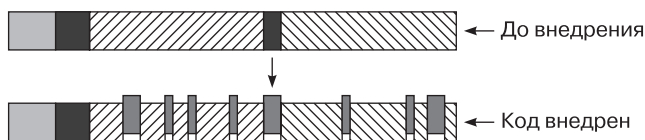


Рис. 4.12. Внедрение кода в регулярные цепочки

Алгоритм внедрения выглядит так: сканируем файл на предмет поиска регулярных последовательностей и отбираем среди них цепочки наибольшей длины, причем сумма их длин должна несколько превышать размеры кода, так как на каждую цепочку в среднем приходится 11 байт служебных данных: четыре байта на стартовую позицию, один байт — на длину, один — на оригинальное содержимое и еще пять байт на машинную команду перехода к другой цепочке.

Разбиваем код на части, добавляя в конец каждой из них команду перехода на начало следующей. Запоминаем начальные адреса, длины и исходное содержимое всех цепочек в импровизированном хранилище, сооруженном либо внутри PE-заголовка, либо внутри одной из цепочек. Если этого не сделать, потом будет невозможно извлечь код из файла. После чего записываем код в выбранные цепочки.

Категория А: внедрение путем сжатия части файла

Внедрение в регулярные последовательности фактически является разновидностью более общей техники внедрения в файл

путем сжатия его части, в данном случае осуществляемое по алгоритму RLE. Если же использовать более совершенные алгоритмы (например, Хаффмена или LZW), то стратегия выбора подходящих частей значительно упрощается. Мы можем сжать кодовую секцию, а на освободившееся место записать наш код. Для компрессии можно использовать функционал, реализованный в самой ОС (аудио, видео-кодеки, экспортеры графических форматов, сетевые функции сжатия и т.д.).

Естественно, упаковка оригинального содержимого секции (или ее части) не обходится без проблем. Во-первых, следует убедиться, что секция вообще поддается сжатию. Во-вторых, предотвратить сжатие ресурсов, таблиц экспорта (импорта) и другой служебной информации, которая может присутствовать в любой подходящей секции файла и кодовой секции в том числе. В-третьих, перестроить таблицу перемещаемых элементов (если, конечно, она вообще есть), исключая из нее элементы, принадлежащие сжимаемой секции и поручая настройку перемещаемых адресов непосредственно самому внедряемому коду (рис. 4.13).

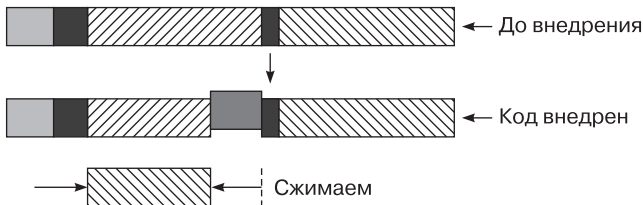


Рис. 4.13. Внедрение кода путем сжатия секции

Распознать факт внедрения в файл путем сжатия части секции трудно, но все-таки возможно. Дизассемблирование сжатой секции обнаруживает некоторое количество бессмысленного мусора,стораживающего опытного исследователя, но зачастую ускользающего от новичка. Также стоит обратить внимание на размеры секций. Если виртуальные размеры большинства секций много больше физических, то файл, по всей видимости, сжат каким-либо упаковщиком.

Категория В: раздвижка заголовка

Когда пространства, имеющегося в РЕ-заголовке (или какой либо другой части файла), оказывается недостаточно для размещения всего кода целиком, мы можем попробовать растянуть заголовок на величину, выбранную по своему усмотрению. До тех пор пока размер за-

головка (SizeOfHeaders) не превышает физического смещения первой секции, такая операция осуществляется элементарно, но вот дальше начинаются проблемы, для решения которых приходится кардинально перестраивать структуру исполняемого файла. Как минимум необходимо увеличить физические адреса начала всех секций на величину, кратную принятой степени выравнивания, прописанной в поле «Физическое выравнивание секций», и физически переместить хвост файла, записав код на освободившееся место.

Максимальный размер заголовка равен виртуальному адресу первой секции, так как заголовок не может перекрываться с содержимым страничного имиджа. Учитывая, что минимальный виртуальный адрес составляет 1000h, а типичный размер заголовка — 300h, мы получаем в свое распоряжение порядка 3 Кбайт свободного пространства, достаточного для размещения практически любого кода (рис. 4.14).

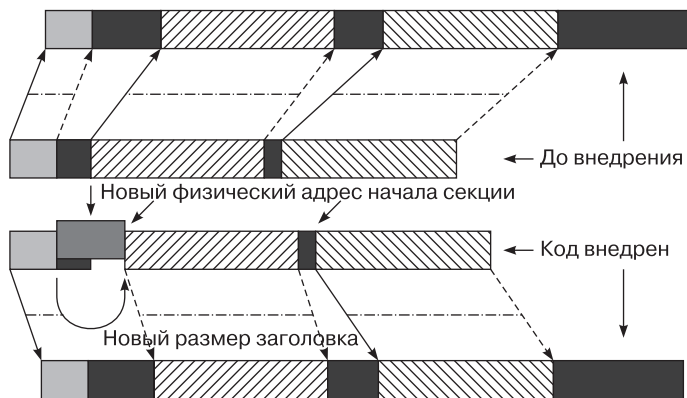


Рис. 4.14. Исполняемый файл и его проекция в память до и после внедрения кода путем раздвижки заголовка

Данный метод внедрения распознается аналогично обычному методу внедрения в PE-заголовок.

Описание программной реализации встраивания ЦВЗ. В программе *Filigrana* использован алгоритм, основанный на встраивании цифрового водяного знака (ЦВЗ) в свободное место исполняемого файла. Подробно данный алгоритм был рассмотрен в предыдущей главе (*Категория А: внедрение в пустое место, в конце секции файла*). В качестве ЦВЗ используется изображение формата BMP.

В общем виде алгоритм встраивания ЦВЗ представлен на рис. 4.15.

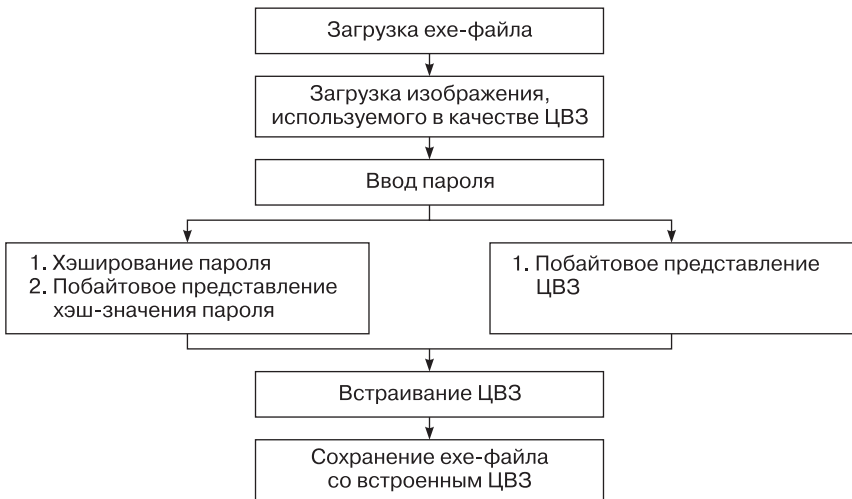


Рис. 4.15. Алгоритм встраивания ЦВЗ

Для демонстрации работы программы встраивания ЦВЗ возьмем два исполняемых файла:

- 1) программу Calc. exe из стандартной поставки Windows, размер программы на диске — 114 Кб (116 376 байт);
- 2) установочный файл SkypeSetup.exe программы Skype, размер файла на диске — 1,88 МВ (1 978 368 байт).

Встроим ЦВЗ наибольшего допустимого размера для каждого из исполняемых файлов.

В нашем случае для первого файла этот размер оказался равным 361 байт, о чем нам и сообщает программа (рис. 4.16).

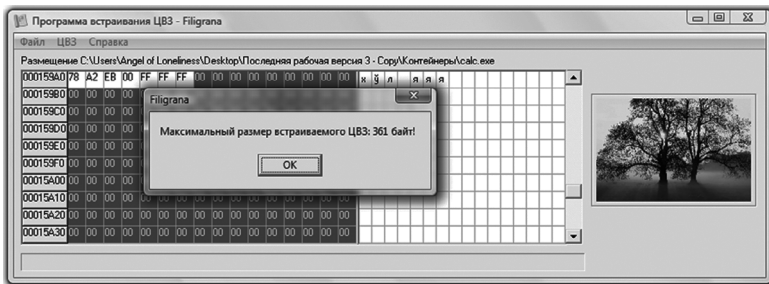


Рис. 4.16. Сообщение о допустимом размере встраиваемого ЦВЗ

Для второго файла возможный размер ЦВЗ для встраивания увеличился до 13 668 байт (рис. 4.17).

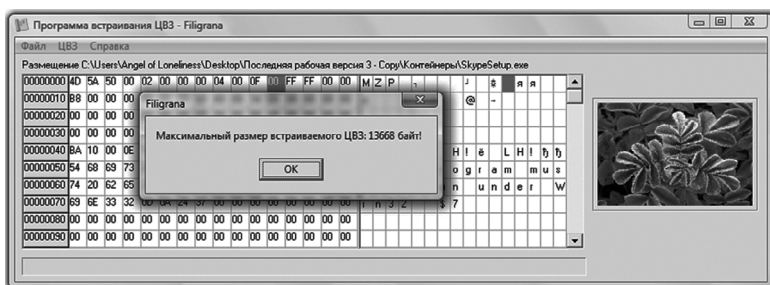


Рис. 4.17. Сообщение о допустимом размере встраиваемого ЦВЗ

При этом в процентном соотношении (размер встраиваемого ЦВЗ к размеру исполняемого файла) первый файл выигрывает у второго $\sim 0,3\%$ против $\sim 0,15\%$. Но, как видно из выходных данных, наиболее эффективно использование программы для встраивания в файлы большего размера, что логично с точки зрения структуры исполняемых файлов, да и выявить факт встраивания в exe-файл большего размера будет сложнее из-за размеров дизассемблированного кода, при этом рекомендуется использовать ЦВЗ небольшого размера относительно размеров контейнера.

После встраивания размеры исполняемых файлов не изменились (рис. 4.18).

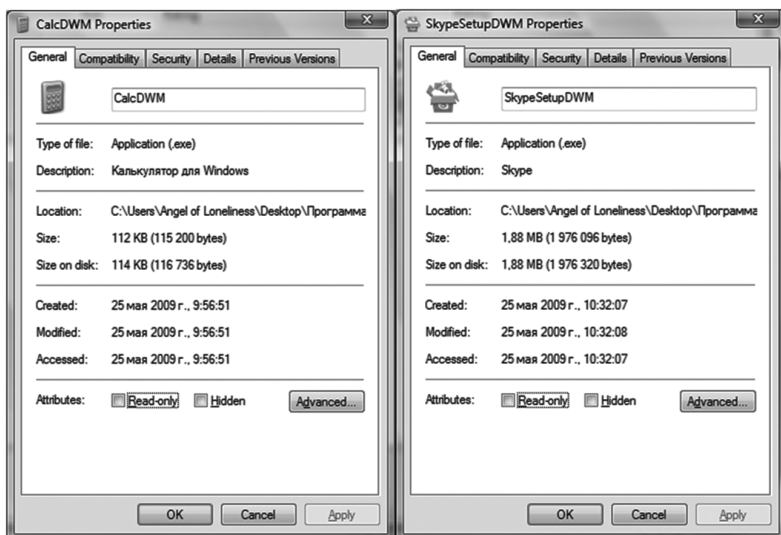
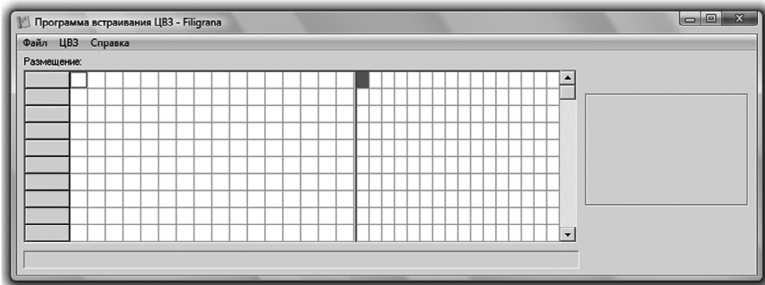


Рис. 4.18. Размеры exe-файлов, содержащих ЦВЗ

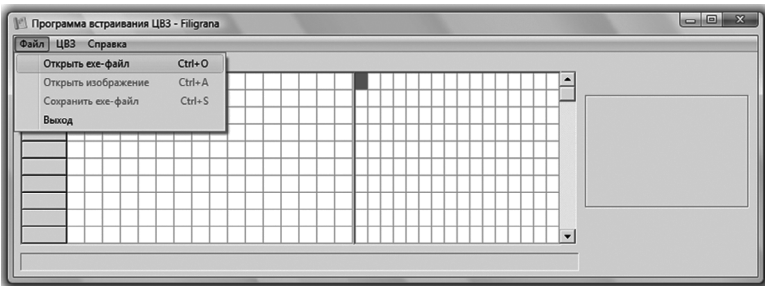
Задание

Встраивание цифрового водяного знака

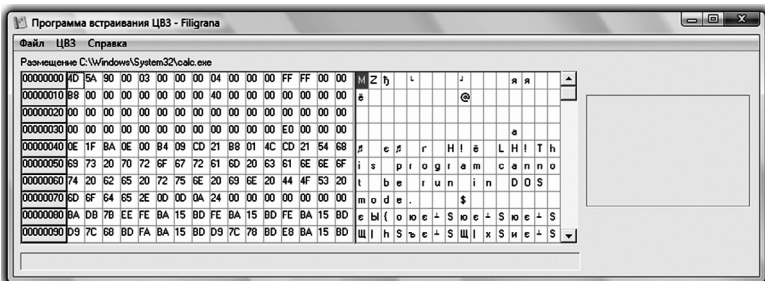
1. Запустите программу *Filigrana*, предварительно установленную на ваш компьютер.



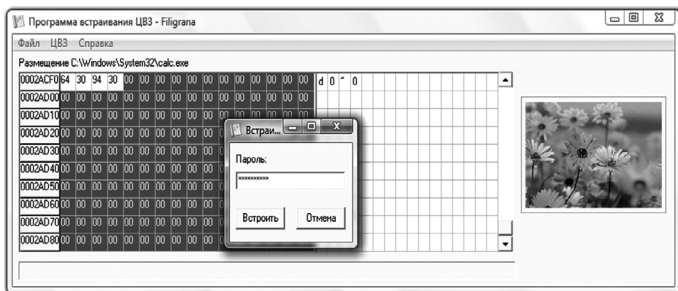
2. Выберите пункт «Открыть ехе-файл» из меню «Файл» появившегося окна или нажмите комбинацию клавиш «Ctrl + O».



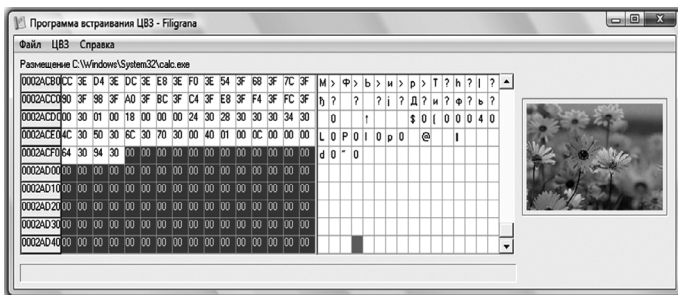
3. В появившемся диалоговом окне выберите необходимый ехе-файл, два раза кликнув на нем левой кнопкой мыши или с помощью кнопки «Открыть». В окне главной формы отобразятся данные о выбранном файле в шестнадцатеричном и ASCII формате.



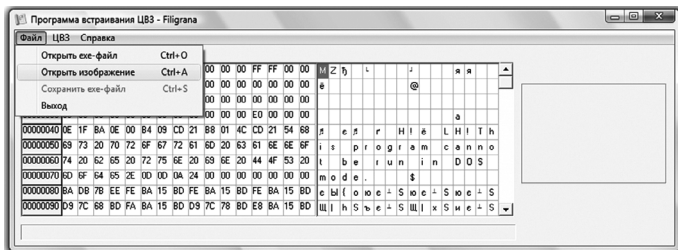
4. Выберите пункт «Открыть изображение» из меню «Файл» или нажмите комбинацию клавиш «Ctrl + A».



5. В появившемся диалоговом окне выберите изображение в формате .bmp, которое вы хотите использовать в качестве ЦВЗ, два раза кликнув на нем левой кнопкой мыши или с помощью кнопки «Открыть». В окне preview главной формы отобразится выбранное изображение, в hex-окне курсор автоматически переместится на строку, с которой начнется последующее встраивание цифрового водяного знака.



6. Чтобы встроить выбранное изображение, введите пароль (используйте сочетание букв, цифр и символов) в окне «Встраивание ЦВЗ», вызываемого из пункта «Встроить изображение» меню «ЦВЗ» главной формы или нажатием комбинации клавиш «Alt + I».



7. После ввода пароля нажмите кнопку «Встроить». В случае если выбранное изображение удовлетворяет условиям встраивания (размер ЦВЗ не превышает размер контейнера и ехе-файл пригоден для встраивания), оно будет встроено, после чего появится соответствующее сообщение, а в hex-окне можно будет просмотреть результаты встраивания.

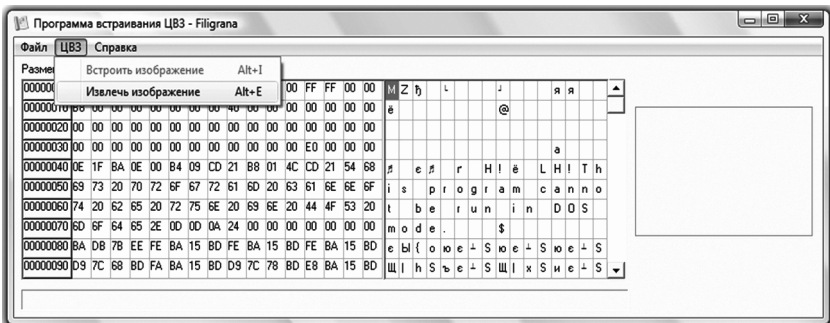


8. Для сохранения исполняемого файла со встроенным ЦВЗ выберите пункт «Сохранить ехе-файл» из меню «Файл» или нажмите комбинацию «Ctrl + S», введите имя сохраняемого файла и нажмите кнопку «Сохранить».

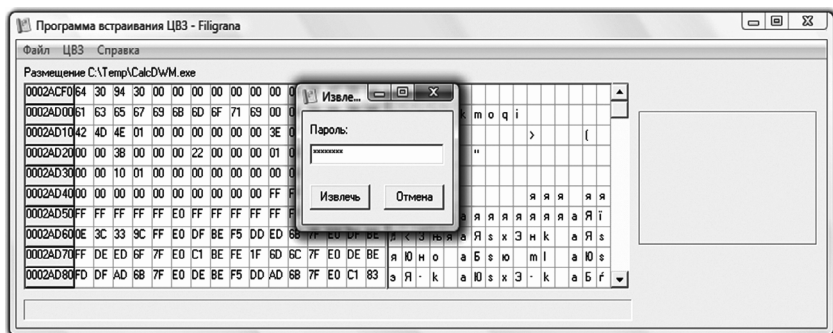
Внимание! Для успешного извлечения ЦВЗ необходимо запомнить свой пароль или сохранить его в месте, недоступном для других пользователей. Никому не показывайте свой пароль!

Извлечение цифрового водяного знака

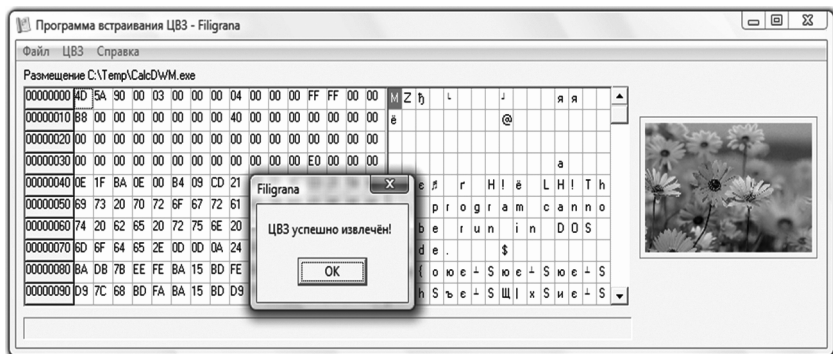
1. Откройте ехе-файл, содержащий ЦВЗ. Выберите пункт «Извлечь изображение» меню «ЦВЗ» главной формы или нажмите комбинацию клавиш «Alt + E».



2. Для извлечения изображения в окне «Извлечение ЦВЗ» введите пароль, который использовался при встраивании цифрового водяного знака, и нажмите кнопку «Извлечь».



3. Если Вы ввели верный пароль, появится соответствующее сообщение, а в окне preview главной формы отобразится изображение, встроенное ранее.



4. Сохраните в отчете экранные формы, демонстрирующие процесс встраивания ЦВЗ в различные исполняемые файлы. Сделайте выводы об эффективности изученного метода защиты.

5. Включите в отчет о лабораторной работе ответы на контрольные вопросы, выбранные в соответствии с номером варианта из табл. 4.1.

Таблица 4.1

Номер варианта	Контрольные вопросы
1, 5, 7, 26	Перечислите способы защиты программных продуктов. Укажите их достоинства и недостатки
2, 4, 6	Какие методы включает юридическая защита программных продуктов? Охарактеризуйте основные из них
11, 13	Перечислите основные международные и отечественные источники защиты прав авторов программ
12, 14, 16	В чем заключается процедура лицензирования программ? Какими нормативными документами регулируется процесс лицензирования программ?
3, 9, 18, 29	Перечислите технические методы защиты программных продуктов. Кратко охарактеризуйте каждый из них
20, 22, 24	Какие методы стеганографии могут использоваться для защиты программных продуктов? Сравните методы стеганографической защиты и технической защиты ПО
10, 17, 19	Какие особенности структуры РЕ-файлов дают возможность эффективного внедрения цифровых водяных знаков?
21, 23, 25	Опишите суть метода внедрения кода в РЕ-файлы за счет размещения кода в свободном месте программы (интеграция)
8, 28, 27	Какой из методов внедрения кода в РЕ-файлы используется в программе <i>Filigrana</i> ? Опишите суть этого метода, его достоинства и недостатки
12, 15, 30	Какие способы обнаружения, извлечения и модификации ЦВЗ вы можете предложить для изученного метода защиты ПО?

ЛАБОРАТОРНАЯ РАБОТА № 14

ЗАЩИТА ЭЛЕКТРОННЫХ ДОКУМЕНТОВ С ИСПОЛЬЗОВАНИЕМ ЦИФРОВЫХ ВОДЯНЫХ ЗНАКОВ

Цель работы: ознакомление с методами защиты электронных документов с использованием цифровых водяных знаков.

Примечание. Для выполнения лабораторной работы на компьютере необходимо установить программы *watermark_1.05* и *watermark_2*.

Описание лабораторной работы. *Цифровые водяные знаки* (ЦВЗ) являются одной из наиболее перспективных областей использования компьютерной стеганографии. На электронные документы наносится специальная метка, которая остается невидимой для глаз, но распознается специальными программами. Такое программное обеспечение уже используется в электронных версиях некоторых журналов. Данное направление стеганографии призвано обеспечить защиту интеллектуальной собственности.

Водяные знаки могут быть как видимыми (или печатными), так и скрытыми (или цифровыми). Печатный водяной знак представляет собой некоторую видимую метку на изображении (рис. 4.19) — как правило, это подпись автора и (или) его логотип. Такие водяные знаки показывают, что данное изображение принадлежит конкретному автору или компании. Однако опытный компьютерный график при желании сможет удалить печатный водяной знак или заменить его на другой.

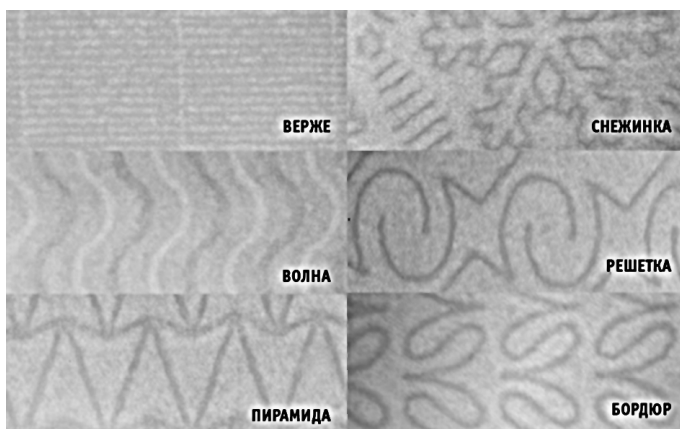


Рис. 4.19. Примеры печатных водяных знаков

Цифровой водяной знак — это цифровой код (как правило, он представляет собой идентификационные данные — ID, URL, адрес электронной почты, логотип и пр.), вставленный в электронный документ, чтобы идентифицировать информацию об авторских правах. В отличие от печатного водяного знака он невидим. ЦВЗ применяется для того, чтобы обеспечить защиту авторских прав для интеллектуальных ресурсов в цифровом формате. При этом биты информации, представляющие водяной знак, разбросаны внутри файла, поэтому они не могут быть идентифицированы или изменены.

ЦВЗ могут быть трех типов: *робастные, хрупкие и полухрупкие*. Под робастностью понимается устойчивость ЦВЗ к различного рода воздействиям на стего.

Хрупкие ЦВЗ разрушаются при незначительной модификации заполненного контейнера. Отличие от средств электронной цифровой подписи заключается в том, что хрупкие ЦВЗ все же допускают некоторую модификацию контента. Это важно для защиты мультимедийной информации, так как законный пользователь может, например, пожелать сжать изображение. Другое отличие заключается в том, что хрупкие ЦВЗ должны отразить не только факт модификации контейнера, но также вид и местоположение этого изменения.

Полухрупкие ЦВЗ устойчивы по отношению к одним воздействиям и неустойчивы по отношению к другим. Вообще говоря, все ЦВЗ могут быть отнесены к этому типу. Однако полухрупкие ЦВЗ специально проектируются так, чтобы быть неустойчивыми по отношению к определенному роду операций. Например, они могут позволять выполнять сжатие изображения, но запрещать вырезку из него отдельных элементов или вставку в него фрагментов.

К цифровым водяным знакам предъявляются следующие требования:

- невидимость; когда на изображение ставится видимый логотип держателя прав, он, как правило, располагается в таком месте, где не сможет быть помехой для зрительного восприятия иллюстрации, следовательно и его умышленное удаление не повлечет за собой особых нарушений ее структурной целостности; исходя из сказанного, для предотвращения обнаружения и удаления метка должна быть невидима и хорошо скрыта данными исходного изображения, не допуская заметного искажения последнего не только из-за возможного обнаружения метки, но и из-за чисто эстетических соображений — не имеет смысла маркировать изображение, если его качество ухудшится настолько, что оно уже в принципе не будет похоже на оригинал;

- ЦВЗ должен быть устойчивым либо неустойчивым к преднамеренным и случайным воздействиям (в зависимости от приложения), если ЦВЗ используется для подтверждения подлинности, то недопустимое изменение контейнера должно приводить к разрушению ЦВЗ (хрупкий ЦВЗ), если же ЦВЗ содержит идентификационный код, логотип фирмы и т.п., то он должен сохраниться при максимальных искажениях контейнера, конечно, не приводящих к существенным искажениям исходного контента; например, у изображения могут быть отредактированы цветовая гамма или яркость, у аудиозаписи — усилено звучание низких тонов и т.д., кроме того, ЦВЗ должен быть робастным по отношению к аффинным преобразованиям изображения, т.е. его поворотам, масштабированию; при этом надо различать устойчивость самого ЦВЗ и способность декодера верно его обнаружить, скажем, при повороте изображения ЦВЗ не разрушится, а декодер может оказаться неспособным выделить его; существуют приложения, когда ЦВЗ должен быть устойчивым по отношению к одним преобразованиям и неустойчивым по отношению к другим, например, может быть разрешено копирование изображения (ксерокс, сканер), но наложен запрет на внесение в него каких-либо изменений;
- защищенность (стойкость к фальсификации); если метка должна быть устойчива к различного вида помехам и трансформациям, следовательно она должна быть устойчива к действиям, направленным на ее удаление;
- ЦВЗ должен иметь низкую вероятность ложного обнаружения скрытого сообщения в сигнале, его не содержащем, в некоторых приложениях такое обнаружение может привести к серьезным последствиям, например, ложное обнаружение ЦВЗ на DVD-диске может вызвать отказ от его воспроизведения плеером;
- ЦВЗ должен вычислительно легко извлекаться законным пользователем;
- должна существовать возможность добавления к стегу дополнительных ЦВЗ, например, на DVD-диске имеется метка о допустимости однократного копирования, после осуществления такого копирования необходимо добавить метку о запрете дальнейшего копирования; можно было бы, конечно, удалить первый ЦВЗ и записать на его место второй, однако это противоречит предположению об устойчивости ЦВЗ к атакам удаления; лучшим выходом является добавление еще одного ЦВЗ, после которого первый не будет приниматься во внимание.

Удовлетворить всем этим требованиям не просто, однако существует множество компаний, предлагающих конкурирующие технологии и соответствующие программы для внедрения цифровых водяных знаков.

Основные области использования технологии ЦВЗ могут быть объединены в три группы: *защита от копирования*, *скрытая аннотация документов* и *доказательство аутентичности информации* (аутентификация) (рис. 4.20).

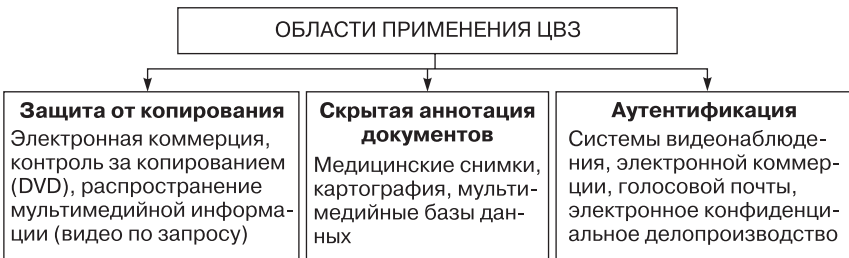


Рис. 4.20. Области применения ЦВЗ

Необходимо отметить, что наибольшие достижения стеганографии в прошедшем десятилетии были получены именно в области развития цифровых водяных знаков. Эти достижения вызваны реакцией общества на актуальнейшую проблему защиты авторских прав в условиях общедоступных компьютерных сетей.

Алгоритмы встраивания ЦВЗ в полутоновые и бинарные изображения. В настоящее время существует не так много алгоритмов, позволяющих встраивать цифровой водяной знак в полутоновые и бинарные изображения. Во многом это связано с тем, что разработчикам приходится выбирать между качеством изображения и робастностью встраиваемой информации. Далее будут рассмотрены основные принципы работы некоторых алгоритмов, встраивающих ЦВЗ в изображения, основными цветами которых являются градации серого.

Алгоритмы скрытия данных в частотной области изображения. Во многих методах скрытия данных в изображениях используется та или иная декомпозиция изображения-контейнера. Среди всех преобразований наибольшую популярность в стеганографии получило дискретное косинусное преобразование (ДКП), что отчасти объясняется его успешным применением при сжатии изображений

Обычно при встраивании информации с помощью ДКП контейнер разбивается на блоки 8×8 пикселей. Преобразование применяется к каждому блоку, в результате чего получаются матрицы коэффи-

циентов ДКП также размером 8×8 (рис. 4.21). Коэффициент в левом верхнем углу матрицы обычно называется ДС-коэффициентом и содержит информацию о яркости всего блока. Остальные коэффициенты называются АС-коэффициентами. Основным недостатком данных алгоритмов является необходимость достаточно больших по размеру контейнеров.

	1	2	3	4	5	6	7	8
1								16
2		НЧ					14	15
3						12	13	
4				9	10	11		
5				7	8			
6			5	6				
7		3	4			ВЧ		
8	1	2						

Рис. 4.21. Матрица ДКП

Алгоритм Хсу и Ву. Основной особенностью данного алгоритма является то, что декодеру ЦВЗ требуется исходное изображение. Однако декодер определяет не факт наличия ЦВЗ, а выделяет встроенные данные. В качестве ЦВЗ выступает черно-белое изображение размером вдвое меньше контейнера (рис. 4.22). Для большей стойкости и скрытности результатов использования данного стеганографического метода количество встраиваемой информации на практике пытаются уменьшить.



Рис. 4.22. Внедряемый ЦВЗ

Перед встраиванием изображение подвергается случайным перестановкам или перестановкам, зависящим от характеристик блоков контейнера, в который будет встроена информация (рис. 4.23). ЦВЗ

встраивается в среднечастотные коэффициенты ДКП (четвертая часть от общего количества). Эти коэффициенты расположены вдоль второй диагонали матрицы ДКП.

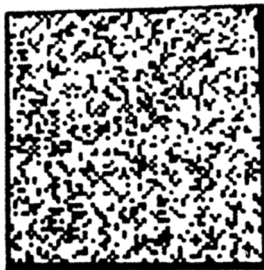


Рис. 4.23. Результат псевдослучайной перестановки элементов ЦВЗ

Для внедрения бита ЦВЗ s_i в коэффициент $c_b(j, k)$ находится знак разности коэффициента текущего блока и соответствующего ему коэффициента из предыдущего блока:

$$d_1(i) = \text{sign}(c_b(j, k) - c_{b-1}(j, k)).$$

Если необходимо встроить 1, коэффициент $c_b(j, k)$ меняют так, чтобы знак разности стал положительным, если 0 — то чтобы знак стал отрицательным.

Графическое представление извлеченного ЦВЗ, встраивание которого производилось путем изменения отношений между значениями ДКП соседних блоков, изображено на рис. 4.24.



Рис. 4.24. ЦВЗ, извлеченный из контейнера

Следует отметить, что этот алгоритм не является робастным по отношению к JPEG-компрессии, так же его отрицательными сторонами являются возможность встраивания ЦВЗ только в виде бинарного изображения и необходимость исходных данных для извлечения ЦВЗ.

Алгоритм Фридрих

Данный алгоритм является композицией двух алгоритмов: в одном данные встраиваются в низкочастотные, в другом — в среднечастотные коэффициенты ДКП. Каскадное применение двух различных алгоритмов приводит к хорошим результатам в отношении робастности. Это объясняется тем, что недостатки одного алгоритма компенсируются достоинствами другого. Исходный сигнал детектору ЦВЗ не требуется.

Перед встраиванием ЦВЗ в низкочастотные коэффициенты изображение преобразуется в сигнал с нулевым математическим ожиданием и определенным отклонением яркости так, чтобы абсолютные значения коэффициентов ДКП попали в определенный диапазон. Для этой цели используется следующее преобразование:

$$I \rightarrow \frac{1024}{\sqrt{XY}} \frac{I - \hat{I}}{\sigma(I)},$$

где X, Y — размеры изображения I в пикселях, $\hat{I}$ и $\sigma(I)$ — соответственно математическое ожидание и стандартное отклонение значений яркости пикселей. ЦВЗ представляет собой сигнал в виде последовательности чисел $\{-1; 1\}$.

Далее на основе геометрической прогрессии действительных чисел

$$t_0 = 1, \quad t_{i+1} = \frac{1+\alpha}{1-\alpha} t_i,$$

где параметр $\alpha \in (0, 1)$, строится функция

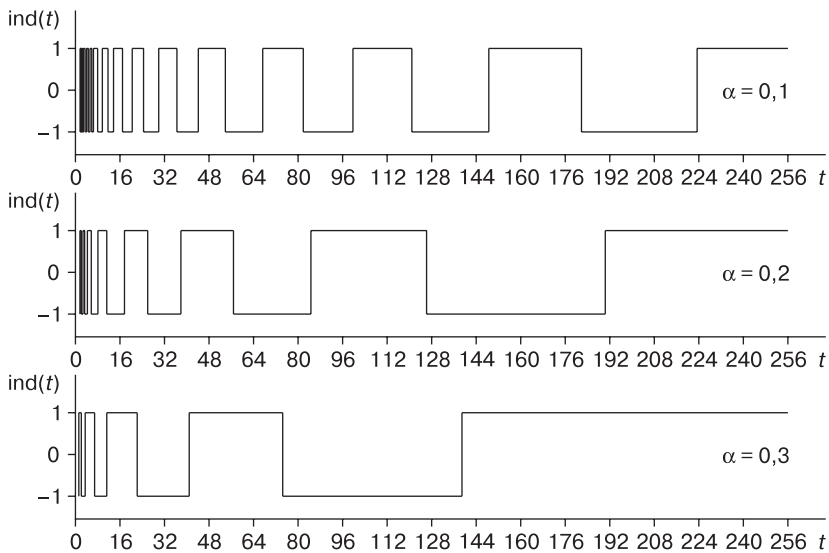
$$\text{ind}(t) = (-1)^i, \quad t > 1, \quad \tau \leq t < \tau_{i+1},$$

позволяющая для каждого вещественного числа $t > 1$ определить его индекс. Этот индекс изменится только в том случае, если к числу t прибавить (отнять) число, превосходящее значение αt . На рисунке 4.25 показан вид функции $\text{ind}(t)$ для $\alpha = 0,1$, $\alpha = 0,2$, $\alpha = 0,3$.

Для внедрения бита ЦВЗ s_i в коэффициент c_j последний изменяется не более чем на 100α процентов так, чтобы $\text{ind}(|c'_j|)s_i$.

Операция извлечения проводится путем выполнения аналогичных с операцией встраивания преобразований контейнера, который подозревается на наличие скрытого сообщения.

В среднечастотные коэффициенты ДКП информация встраивается путем умножения преобразованного значения ЦВЗ на параметр α и сложения результата со значением коэффициента. Предварительное кодирование ЦВЗ выполняется по следующему алгоритму.

Рис. 4.25. Индексная функция $\text{ind}(t)$

Вход алгоритма: сообщение длины M , состоящее из символов $m_i \in \{1, \dots, B\}$.

Выход алгоритма: ЦВЗ длины N , состоящий из вещественных чисел s_i .

Для кодирования символа m_i генерируется $N + B + 1$ чисел псевдослучайной последовательности $r_i \in \{-1, 1\}$. Эту последовательность будем называть i -м случайным вектором.

Первые m_i чисел этого вектора пропускаются, а следующие N чисел образуют вектор V_i , используемый при дальнейшем суммировании.

Для каждого символа сообщения генерируются статистически независимые различные случайные вектора.

В качестве ЦВЗ используется сумма векторов V_i .

В результате композиции встраивания в НЧ и СЧ-коэффициенты получился алгоритм, достаточно стойкий ко многим атакам. Такой алгоритм полезен для защиты произведений искусства и других полutoновых изображений, являющихся авторской собственностью.

Алгоритм В.А. Мутекина. Рассматриваемый метод основан на кодировании встраиваемой информации количеством темных или светлых пикселей в блоке фиксированного размера. В данном случае используется кодирование одного бита встраиваемой информации битом четности блока 3×3 пикселя.

Предположим, что имеется бинарное изображение размера $M \times N$ пикселей. Разделим его на множество непересекающихся блоков B_i , каждый размера 3×3 , где $1 \leq i \leq (M \times N) : (3 \times 3)$. В каждый из этих блоков будет внедрен бит информации s_i . Двумерный генератор псевдослучайных точек выбирает блок для внедрения. Решение о необходимости изменения бита в данном блоке принимается на основе формулы «контроля четности»:

$$H = \begin{cases} 0, s_i = \sum_{j=1}^9 p_j \pmod{2} \\ 1, s_i \neq \sum_{j=1}^9 p_j \pmod{2} \end{cases},$$

где p_j — значение функции яркости пикселя в рассматриваемом блоке B_i .

Если внедряемый бит соответствует биту четности H данного блока, то считается, что бит информации уже внедрен. Иначе необходимо инвертировать один пиксель в блоке. Выбор пикселя-кандидата для внедрения бита ЦВЗ основан на преобладающем цвете в блоке.

$$\text{Цвет кандидата} = \begin{cases} \text{Черный, если } N_B < 4 \\ \text{Белый, если } N_B \geq 7 \\ \text{Любой, если } 4 \leq N_B < 7 \end{cases},$$

где N_B — количество темных пикселей в выбранном блоке.

Предположим, что существует L кандидатов в блоке, из которых нужно выбрать один. На окрестность каждого кандидата накладывается матрица веса размера 5×5 , с помощью которой рассчитываем «вес» каждого кандидата.

Предположим, что кандидат имеет координаты (m, n) . Тогда вес r рассчитывается следующим образом:

$$r(m, n) = \sum_{i=-2}^2 \sum_{j=2}^2 (\omega(i, j) \times I_{mn}(i, j));$$

$$I_{mn}(i, j) = \begin{cases} 0, & \text{если } O(m, n) = O(m+i, n+j) \\ 1, & \text{если } O(m, n) \neq O(m+i, n+j) \end{cases},$$

где $O(m, n)$ — значение функции яркости пикселя исходного изображения с координатами (m, n) ;

$$W = (\omega_{i,j}) = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 2 & 1 & 0 \\ 1 & 2 & 0 & 2 & 1 \\ 0 & 1 & 2 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \end{bmatrix} \quad \text{— матрица веса.}$$

Вес $r(m, n)$ является мерой, характеризующей взаимосвязь кандидата и его соседних пикселей. Если найдено несколько кандидатов с одинаковым весом, то выбирается любой из них. В результате встраивания значение кандидата с наибольшим весом изменяется на противоположное. Таким образом, один блок содержит один бит ЦВЗ.

На рисунке 4.26 представлены увеличенный фрагмент оригинального изображения и фрагмент того же самого изображения после внедрения в него ЦВЗ объемом 40 бит.



Рис. 4.26. Пример встраивания ЦВЗ по алгоритму Митекина

Рассмотренный метод кодирования является наиболее широко распространенным для встраивания ЦВЗ в бинарные изображения. На его основе разработан ряд модифицированных алгоритмов.

Одной из модификаций является расширение данного метода на полутоновые изображения. Изображение-контейнер представляет собой совокупность битовых «слоев», каждый из которых представляет изображение с глубиной цвета 1 бит. Таким образом, данный алгоритм применяется для одного или нескольких выбранных «слоев». Выбранный слой изображения модифицируется в соответствии с представленным выше алгоритмом. Результатом встраивания ЦВЗ является изображение той же глубины цвета, что и исходное.

Недостатком данного алгоритма является низкая степень сохранения визуального качества изображения. Во-первых, это связано с ма-

лыми размерами блоков для встраивания информации. Во-вторых, с выбором блоков для встраивания, ведь в зависимости от выбранного алгоритма встраивание может производиться и в полностью белые и в полностью черные блоки, что приведет к значительному искажению изображения-контейнера. В модификациях данного алгоритма предприняты попытки устранить данные недостатки.

Модификации алгоритма В.А. Митекина. Для программного обеспечения, встраивающего ЦВЗ в полутоновые изображения были разработаны две модификации алгоритма, описанного в работе В.А. Митекина. Обе модификации направлены на то, чтобы встроенная информация была незаметна в изображении-контейнере, т.е. на то, чтобы сделать алгоритм более робастным к атакам обнаружения.

Если встраивать биты информации во все подряд блоки изображения, то оно будет сильно искажено, а сам ЦВЗ будет легко обнаружен.

Было предложено следующее: в качестве ключа используется матрица K :

$$K = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}.$$

Алгоритм обработки, написанный на псевдокоде, выглядит следующим образом:

```

If ( $0 < \text{sum}(F_i \wedge K) < \text{sum}(K)$ )
  If ( $\text{sum}(F_i \wedge K) \bmod 2 = b$ )
    Keep  $F_i$  intact;
  Else if ( $\text{sum}(F_i \wedge K) \bmod 2 = 1$ )
    Randomly pick a bit  $[F_i]_{j,k} = 0$ ,  $[K]_{j,k} = 1$  and change  $[F_i]_{j,k}$  to 1;
  Else if ( $\text{sum}(F_i \wedge K) = \text{sum}(K) - 1$ )
    Randomly pick a bit  $[F_i]_{j,k} = 1$ ,  $[K]_{j,k} = 1$  and change  $[F_i]_{j,k}$  to 0;
  Else
    Randomly pick a bit  $[F_i]_{j,k}$ ,  $[K]_{j,k} = 1$  and complement  $[F_i]_{j,k}$ ;
  Else
    No data will be embedded in  $F_i$  and  $F_i$  keeps intact
  
```

На рисунке 4.27 изображен пример встраивания трех бит информации в черно-белое изображение размером 6×6 . Изображение F делится на четыре равных блока размером 3×3 , и в них по порядку встраивается информация.

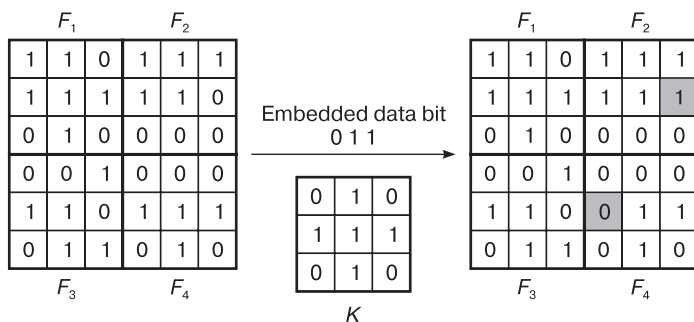


Рис. 4.27. Пример встраивания информации с использованием ключа.

F_1 : $\text{sum}(F_1 \wedge K) = 5 = \text{sum}(K)$ — в этот блок данные не встраиваются;

F_2 : $\text{sum}(F_2 \wedge K) = 3$ — в этот блок можно встроить 1 бит данных, встраиваемый бит — 0, F'_2 — результат встраивания в соответствии с описанным выше алгоритмом;

F_3 : $\text{sum}(F_3 \wedge K) = 3$ — в этот блок можно встроить 1 бит данных, встраиваемый бит — $1 = F_3 \bmod 2$, блок контейнера изменять не нужно;

F_4 : $\text{sum}(F_4 \wedge K) = 3$ — в этот блок можно встроить 1 бит данных, встраиваемый бит — 1, F'_4 — результат встраивания в соответствии с описанным выше алгоритмом.

Используя описанный выше алгоритм для определения, необходимы ли изменения в каком-либо конкретном блоке для встраивания бита информации, дальше действуем в соответствии с алгоритмом Митекина. Вместо случайного выбора бита для изменения выбираем наиболее подходящий бит для встраивания посредством наложения на кандидаты матрицы веса.

Недостатком этой модификации, как и исходного алгоритма, является маленький размер блоков, на которые делится изображение-контейнер. Кроме того, ключ, при помощи которого выбираются блоки для встраивания, позволяет избежать встраивания только в полностью белые блоки. Если изображение содержит крупные области черного цвета, то в заполненном контейнере на этом месте вследствие встраивания появятся строки белых пикселей, что нанесет существенный ущерб как внешнему виду изображения, так и робастности ЦВЗ к атакам обнаружения.

С учетом перечисленных выше недостатков, был разработан еще один алгоритм, который является усложненной модификацией первых двух. Во-первых, в нем изображение уже разделяется не на бло-

ки 3×3 , а на блоки 7×7 пикселей. Во-вторых, используется не один, а два ключа, что делает алгоритм выбора блоков сложнее, а самое главное, позволяет избежать встраивания как в полностью белые, так и в полностью черные блоки. Оба этих изменения в совокупности довольно значительно повышают робастность встраиваемого ЦВЗ к атакам обнаружения и степень сохранения визуального качества изображения.

Недостатком данного алгоритма является его ресурсозатратность. Так, для встраивания одного бита информации нам необходим блок размером 7×7 . Таким образом, для встраивания одного символа необходимо восемь блоков или $49 \times 8 = 392$ пикселя. Кроме того, каждый ЦВЗ содержит символ-признак его начала и окончания. Получается, что для встраивания цифрового водяного знака, состоящего только из одного символа, требуется 24 блока 7×7 , т.е. изображение, содержащее 1176 пикселей. И это — минимум, основанный на предположении о том, что все блоки будут пригодны для встраивания (пройдут проверку ключами).

Описание программного обеспечения для встраивания ЦВЗ. На основе модификаций алгоритма В.А. Митекина, описанных выше, предлагаются две программы с одинаковым интерфейсом, реализующие встраивание ЦВЗ двумя разными вариантами одного метода.

Описание структур и алгоритмов. Программы работают с файлами формата .bmp. Этот формат был выбран из-за его избыточности и возможности работать с ним в любой версии операционной системы (ОС) Windows за счет специальных встроенных функций. Программы позволяют встраивать цифровой водяной знак в монохромные изображения.

Обобщенный алгоритм работы программ представлен на рис. 4.28.

Описание интерфейса. Интерфейс обеих программ совершенно одинаков. Так же ничем не отличаются и основы работы с ними. Основные различия заключаются в принципах встраивания ЦВЗ, и, следовательно, как будет показано далее, в результатах встраивания.

Структура главного меню изображена на рис. 4.29.

Пункты меню File позволяют открывать изображения для встраивания (извлечения) ЦВЗ и сохранять модифицированные изображения. Пункты меню Edit запускают процессы встраивания и извлечения цифрового знака, Их дублируют кнопки Hide и Extract, расположенные под строкой ввода текста. В текстовой строке содержится либо текст встраиваемого ЦВЗ, набранный пользователем, либо текст извлеченной метки.

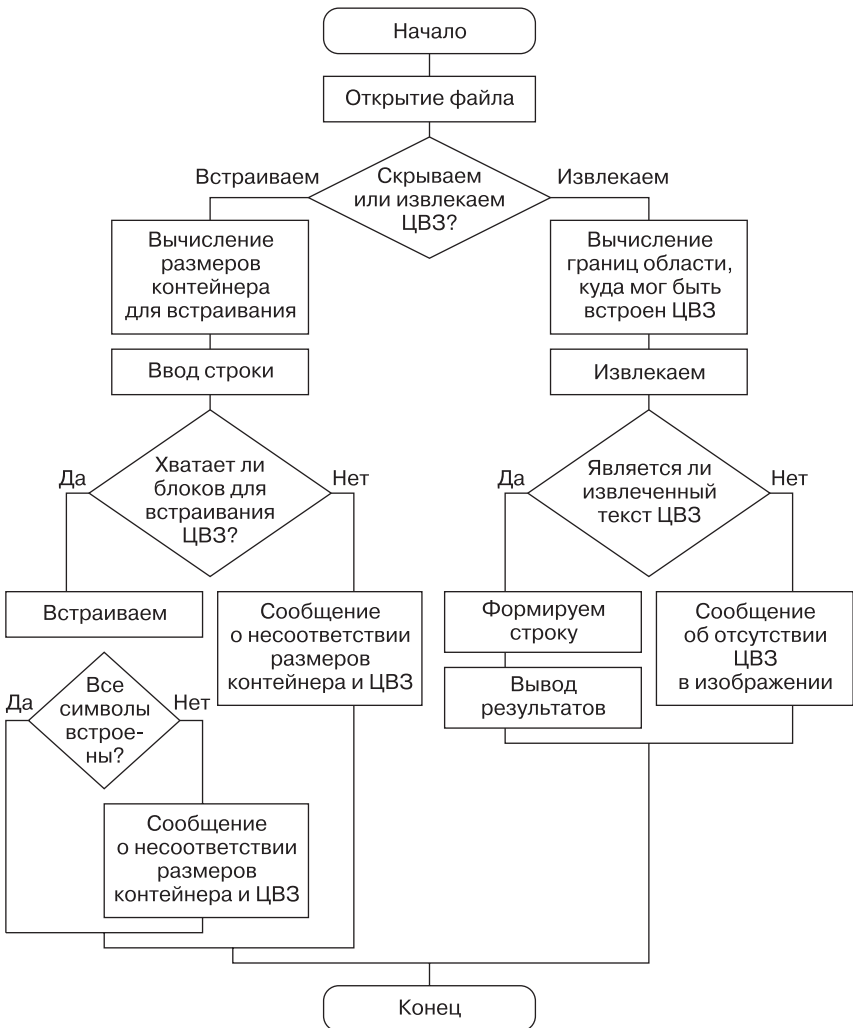


Рис. 4.28. Обобщенный алгоритм работы ПО, встраивающего ЦВЗ

Руководство по использованию ПО и примеры работы. Для работы с любой из программ прежде всего необходимо открыть подходящий файл .bmp. Для этого выбирается пункт меню File — Open и открывается диалоговое окно открытия файла.

Если выбранный файл представляет собой не черно-белое изображение, пользователю выдается сообщение об ошибке (рис. 4.30).

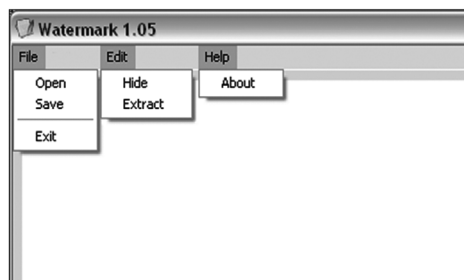


Рис. 4.29. Структура главного меню



Рис. 4.30. Сообщение о неподходящем формате открываемого файла

После открытия изображения пользователь выбирает — извлекаться будет ЦВЗ или встраиваться. Для встраивания в текстовую строку приложения должен быть введен текст метки (рис. 4.31).

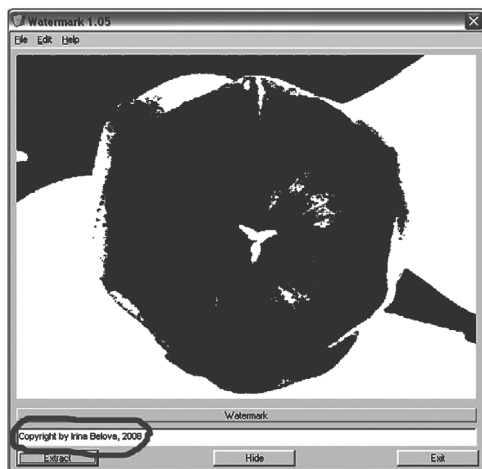


Рис. 4.31. Цифровой водяной знак

После нажатия кнопки Hide или выбора соответствующего пункта меню начинается сам процесс встраивания. В случае если изображение-контейнер слишком мало для встраивания введенной в текстовой строке информации, выдается сообщение об ошибке (рис. 4.32). Если же процесс встраивания завершится успешно, модифицированное изображение будет выведено на экран и пользователь сможет его сохранить. Для этого выбирается пункт меню File — Save.

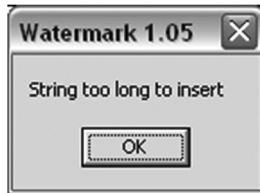


Рис. 4.32. Сообщение о том, что введенный ЦВЗ превышает допустимый для встраивания размер

Для извлечения цифрового водяного знака необходимо нажать кнопку Extract или так же, как и при встраивании, выбрать соответствующий пункт меню. Если в изображении не будет обнаружено скрытой информации, пользователь получит об этом сообщение (рис. 4.33), иначе в текстовой строке будет выведен текст ЦВЗ (рис. 4.34).



Рис. 4.33. Сообщение о том, что ЦВЗ не обнаружен

Обе программы имеют свои преимущества и недостатки. Например, программа, встраивающая ЦВЗ в блоки изображения размера 3×3 , имеет преимущества при работе с изображениями, размер которых не позволяет встроить большое количество символов. Для встраивания одного символа данной программе требуется всего 72 пикселя изображения, в то время как другой программе нужно 392 пикселя. Примеры встраивания ЦВЗ, состоящего из 28 символов (224 бита), обеими программами в изображение размерами 128×117 пикселей показаны рис. 4.35 и 4.36.

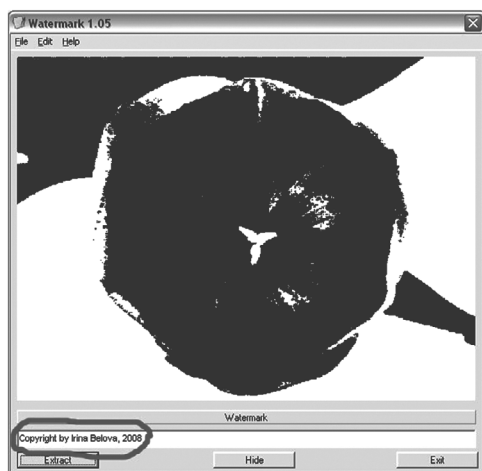
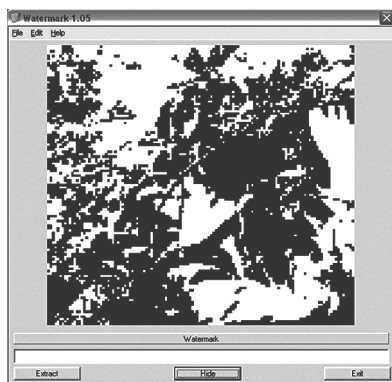


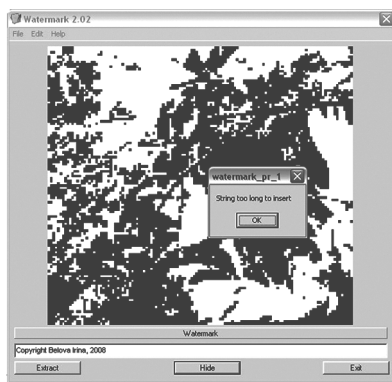
Рис. 4.34. Пример извлечения ЦВЗ



Рис. 4.35. Пустой контейнер размером 128×117 пикселей



а



б

Рис. 4.36. Пример встраивания ЦВЗ в изображение размером 128×117 пикселей: а) в блоки 3×3 пикселя; б) в блоки 7×7 пикселей

При работе с изображениями, имеющими достаточно большие по размеру области черного цвета, лучше использовать программу, работающую с блоками изображения размером 7×7 пикселей. Программа, встраивающая метку в блоки 3×3 пикселя, в связи с наличием всего одного ключа показывает на таких изображениях недостаточно хорошие результаты по сохранению внешнего вида исходного изображения. Примеры работы программ для таких изображений представлены на рис. 4.37 и рис. 4.38. В изображение размером 400×300 пикселей встраивается ЦВЗ из 28 символов.

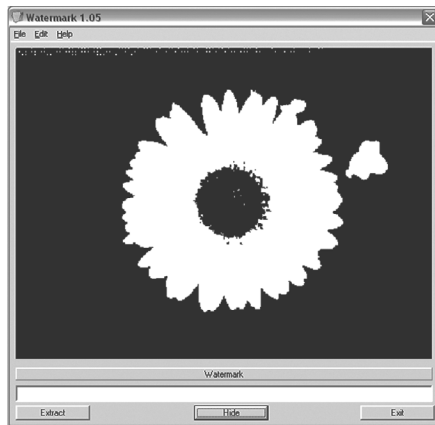


Рис. 4.37. Встраивание ЦВЗ в блоки изображения размером 3×3 пикселя

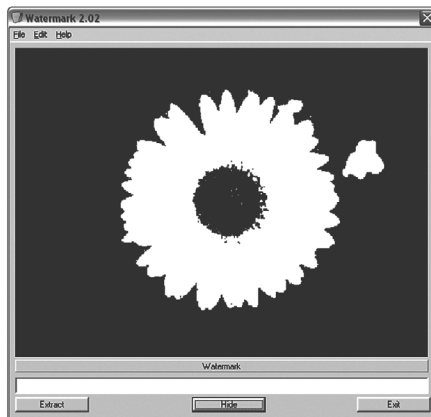


Рис. 4.38. Встраивание ЦВЗ в блоки изображения размером 7×7 пикселей

Задание

1. Запустите программу **watermark_1.05**, предварительно установленную на ваш компьютер.

2. Выберите пункт **Open** из меню **File** появившегося окна. Откройте, подготовленный заранее .bmp файл-контейнер с черно-белым изображением, в который необходимо встроить ЦВЗ (заранее необходимо подготовить три файла-контейнера: с большими областями белого цвета; с большими областями черного цвета; с чередующимися областями белого и черного цветов).

После открытия изображения необходимо выбирать — извлекаться будет ЦВЗ или встраиваться. Для встраивания в текстовую строку приложения должен быть введен текст метки (см. рис. 4.31). Подготовьте текст встраиваемого ЦВЗ и выполните встраивание. Сохраните файл со встроенной меткой в личной папке.

3. Действия, описанные в п. 2, повторите со всеми подготовленными файлами-контейнерами, не изменяя метку встраивания. Сравните полученные файлы-контейнеры на наличие визуальных искажений и по объему с исходными (непомеченными) файлами. Сделайте выводы об эффективности используемого алгоритма встраивания ЦВЗ.

4. Проведите извлечение ЦВЗ из всех помеченных файлов (процесс извлечения описан в разделе 3, см. рис. 4.32, 4.33). Сравните их качество, сделайте выводы об эффективности алгоритма извлечения ЦВЗ.

5. Помеченные файлы-контейнеры подвергните обработке с использованием:

- алгоритмов архивации с потерями;
- преобразований изображения (модификация, обрезание краев, масштабирование).

6. Повторите действия п. 4 по извлечению ЦВЗ из обработанных файлов-контейнеров. Сделайте выводы о робастности ЦВЗ.

7. Запустите программу **watermark_2.02**, предварительно установленную на ваш компьютер. Прodelайте п. 2—5 с использованием программы **watermark_2.02**. Сравните результаты с полученными при использовании программы **watermark_1.05**. Сформулируйте выводы.

8. Сохраните в отчете экранные формы, демонстрирующие процесс встраивания ЦВЗ в различные файлы-контейнеры. Сделайте выводы об эффективности изученного метода защиты.

9. Включите в отчет о лабораторной работе ответы на контрольные вопросы, выбранные в соответствии с номером варианта из табл. 4.2.

Таблица 4.2

Номер варианта	Контрольные вопросы
1, 5, 7, 26	Перечислите способы защиты цифровых графических изображений от модификации и несанкционированного использования. Укажите их достоинства и недостатки
2, 4, 6	В чем особенность робастных, хрупких и полухрупких цифровых водяных знаков? Каковы ограничения в их использовании для защиты электронных документов?
11, 13	Перечислите основные международные и отечественные источники защиты прав авторов цифрового графического контента
12, 14, 16	Алгоритм Хсу и Ву — особенности реализации, достоинства и недостатки
3, 9, 18, 29	Алгоритм Фридрих — особенности реализации, достоинства и недостатки
20, 22, 24	Какие методы стеганографии могут использоваться для защиты графических файлов? Сравните методы стеганографической защиты и криптографической защиты
10, 17, 19	Алгоритм В.А. Митекина — особенности реализации, достоинства и недостатки
21, 23, 25	В чем заключается усовершенствование алгоритма В.А. Митекина в программных продуктах, использованных в лабораторной работе? Оцените эффективность исследованного алгоритма для встраивания ЦВЗ?
8, 28, 27	Подсчитайте максимальных объем встраиваемого ЦВЗ в бинарное изображение размером 1024×128 пикселей: а) в блоки 3×3 пикселя, б) в блоки 7×7 пикселей
12, 15, 30	Подсчитайте максимальных объем встраиваемого ЦВЗ в полутоновое изображение (256 градаций серого) размером 512×256 пикселей: а) в блоки 3×3 пикселя, б) в блоки 7×7 пикселей

ЛАБОРАТОРНАЯ РАБОТА № 15

СТЕГОКОМПЛЕКСЫ, ДОПУСКАЮЩИЕ ИСПОЛЬЗОВАНИЕ АУДИОКОНТЕЙНЕРОВ, НА ПРИМЕРЕ ПРОГРАММЫ *INVISIBLE SECRETS-4*

Цель работы: изучение современных программных средств стеганографии на примере пакета *Invisible Secrets-4*.

Примечание. Для выполнения лабораторной работы на компьютере необходимо установить программу *Invisible Secrets-4*.

Описание лабораторной работы. Пакет *Invisible Secrets-4* отличается проработанностью пользовательского интерфейса, выполненного в виде «мастера», интегрированность в среду *Microsoft Windows* и удобство использования. Данный пакет программ может использовать в качестве контейнеров как аудио, так и графические файлы.

В качестве контейнеров данный программный продукт использует неупакованные аудиоданные, оцифрованные с разрядностью 8 или 16 бит на отсчет. Перед сокрытием сообщения оно шифруется. Доступны такие алгоритмы шифрования, как *Rijndael*, ГОСТ 28147—89, *Blowfish* и др.

При разработке стеганографических методов сокрытия информации основной целью является построение помехоустойчивого, необнаруживаемого метода. В связи с этим при рассмотрении программного продукта уделим особое внимание не интерфейсу, а реализованным методам.

Поскольку не для всех программных продуктов доступен исходный текст программы, то выводы часто приходится делать на основе анализа работы программы на тестовых контейнерах. В качестве тестовых контейнеров были выбраны: тональный сигнал с частотой 440 Гц и тишина (сигнал, у которого все отсчеты равны нулю). На рисунке 4.39 показан результат работы программы *Invisible Secrets-4* на тестовых контейнерах при оцифровке 8 бит на отсчет.

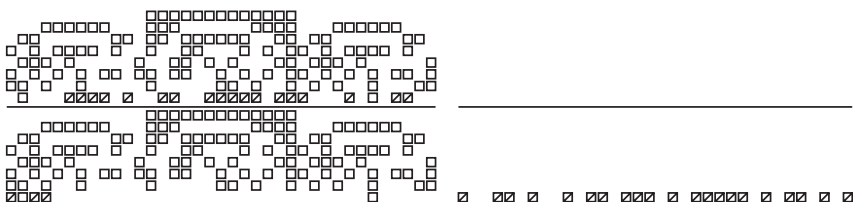


Рис. 4.39. Результат работы программы на тестовых контейнерах

На рисунке по горизонтали указываются отсчеты сигнала, по вертикали — разряды при записи отсчетов в двоичном виде. При наличии прямоугольника в соответствующем разряде соответствующего сэмпла стоит единица, в противном случае ноль. В верхней половине рисунка (выше разделительной линии) изображены отсчеты исходного сигнала контейнера, в нижней половине — отсчеты модифицированного сигнала контейнера. Перечеркнутые прямоугольники соответствуют несовпадающим битам в исходном и модифицированном контейнерах. На левом рисунке показан тональный сигнал, на правом — сигнал, содержащий тишину.

Программа *Invisible Secrets-4* при внедрении данных в аудио контейнер использует наиболее распространенный в настоящее время метод модификации младшего бита (LSB-метод). Более того, при внедрении не производится никакого анализа содержимого контейнера, что видно из рисунка, где программа внедрила информацию в файл, содержащий тишину. Существенным недостатком данного программного продукта также является то, что при внедрении небольшого количества информации вся информация внедряется в начало контейнера, а не распределяется равномерно по всему контейнеру. Таким образом, возможно обнаружение наличия скрытого сообщения на основе анализа локальных различий статистик в разных частях контейнера.

На рисунке 4.40 приведен вид главного меню программы *Invisible Secrets-4*.



Рис. 4.40. Главное меню программы *Invisible Secrets-4*

Программа *Invisible Secrets-4* состоит из шести функциональных модулей. Рассмотрим каждый из них отдельно.

Стеганография

Программа позволяет зашифровать и спрятать данные в таких местах, которые на поверхности выглядят совершенно безобидными, например в картинку, файлы с музыкой или веб-страницы. Такие типы контейнеров — прекрасная маскировка для секретной информации.

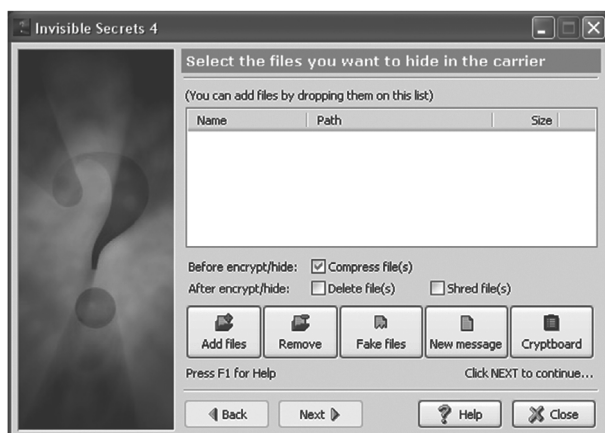


Рис. 4.41. Выбор файла, который необходимо встроить или зашифровать, и задание соответствующих параметров

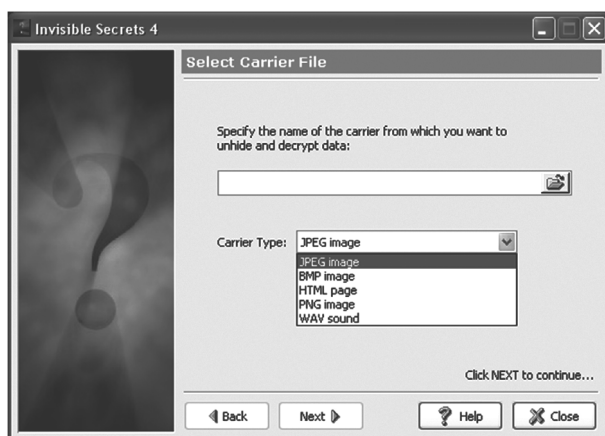


Рис. 4.42. Выбор файла для извлечения из файла-контейнера или расшифрования

Шифрование

Шифрование. В программе *Invisible Secrets-4* предусмотрена защита файлов с использованием современных методов симметричного шифрования. Только владелец секретного пароля сможет прочесть их. Программа поддерживает такие алгоритмы шифрования, как: *Rijndael*, ГОСТ 28147—89, *Blowfish*, *Twofish*, *RC4*, *Cast128*, *Diamond 2*, *Sapphire II*. На рисунке 4.43 приведена панель шифрования файлов, а на рис. 4.44 панель расшифрования файлов.

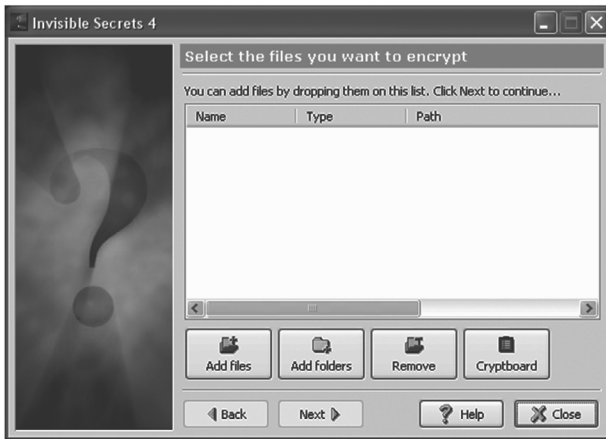


Рис. 4.43. Панель шифрования файлов

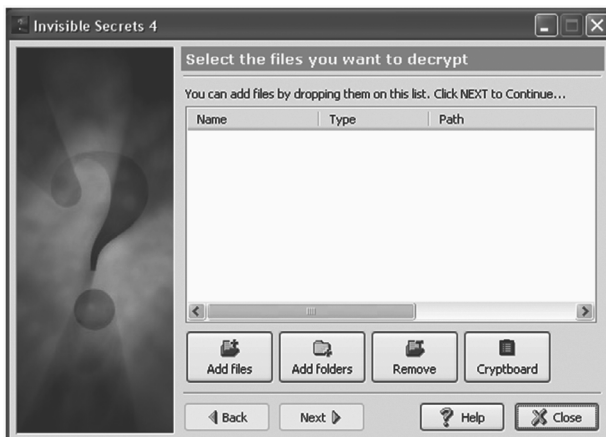


Рис. 4.44. Панель расшифрования файлов

Шифрование почтовых сообщений. Чрезвычайно актуальными являются процессы обмена конфиденциальной информацией в сети. При обмене личными сообщениями или коммерческими секретами возникает необходимость защищать конфиденциальную информацию от несанкционированного прочтения или модификации.. Программа *Invisible Secrets-4* поможет отослать зашифрованное сообщение по *e-mail*, а получателю для его расшифрования необходимо будет знать только правильный пароль.

На рисунке 4.45 приводится панель программы *Invisible Secrets-4* для создания самораспаковывающегося пакета для его отправки по *e-mail*.

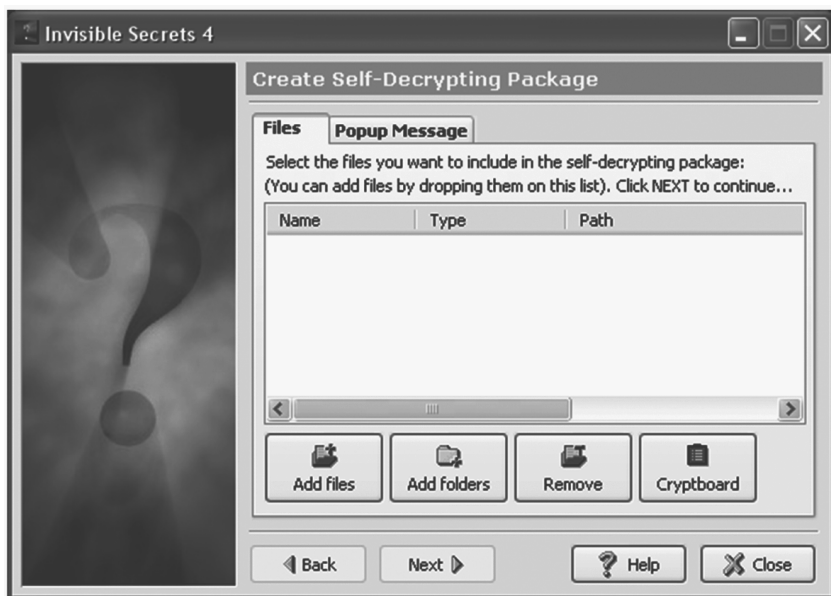


Рис. 4.45. Создание самораспаковывающегося пакета для его отправки по *e-mail*

Уничтожение папок, файлов и ссылок. Для уничтожения секретных папок или файлов без возможности их восстановления, а также различных ссылок в программе *Invisible Secrets-4* используется модуль *DoD 5220.22-M Shredder*. Панель для уничтожения папок и файлов приведена на рис. 4.46, а для уничтожения ссылок на рис. 4.47.



Рис. 4.46. Выбор папки или файла, которые необходимо уничтожить

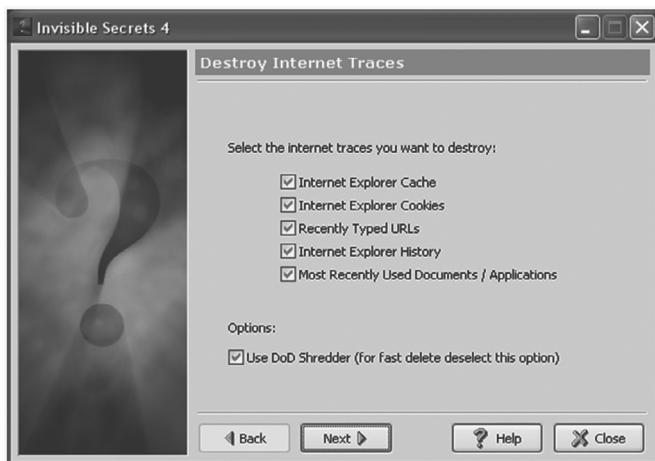


Рис. 4.47. Уничтожение ссылок

Передача паролей

Безопасная передача паролей. Одна из главных проблем, возникающих при организации защищенного обмена информацией, — это передача секретных паролей и ключей. *Invisible Secrets-4* позволяет обмениваться паролями между двумя компьютерами при помощи создания безопасного соединения *IP-к-IP*.

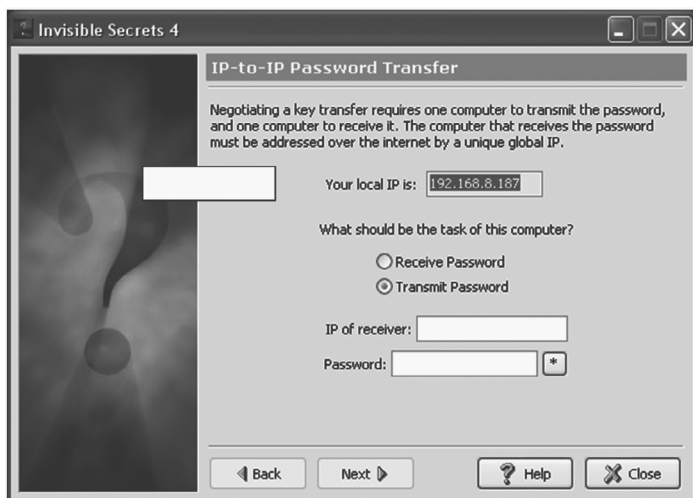


Рис. 4.48. Безопасная передача пароля

Задание

1. Установить на своем компьютере программу *Invisible Secrets-4*.
2. Ознакомиться со сведениями о программе *Invisible Secrets*, изложенными в разделе 1.
3. Запустить программу *Invisible Secrets-4*.
4. Изучить работу программы *Invisible Secrets* во всех шести возможных режимах:
 - стеганография;
 - шифрование;
 - шифрование почтовых сообщений;
 - уничтожение файлов;
 - передача паролей;
 - ограничение доступа к приложениям.
5. Сохранить в отчете экранные формы, демонстрирующие возможности программы *Invisible Secrets-4* при работе в каждом из шести режимов.
6. Сделать выводы об эффективности используемого стеганографического пакета для каждого из рассмотренных режимов.
7. Включить в отчет о лабораторной работе ответы на контрольные вопросы, выбранные в соответствии с номером варианта из табл. 4.3.

Таблица 4.3

Номер варианта	Контрольные вопросы
1, 5, 7, 3, 9, 18, 28	Сформулируйте основные отличия стеганографических и криптографических методов защиты информационных ресурсов. В чем достоинства и недостатки каждого из методов?
2, 4, 6, 8, 20, 22, 24, 26, 30	Перечислите шесть основных режимов работы программы <i>Invisible Secrets-4</i> . В каких случаях вы можете порекомендовать использование каждого из режимов?
11, 13, 15, 10, 17, 19, 27	Оцените эффективность работы программы <i>Invisible Secrets-4</i> с графическими и аудиофайлами различных типов. Для этого в режиме «стеганография» встройте данные в контейнеры различных форматов, сравните пустые и заполненные контейнеры, сделайте выводы
12, 14, 16, 21, 23, 25, 29	При встраивании данных в режиме «стеганография» используйте различные алгоритмы для шифрования встраиваемых данных. Сравните пустые и заполненные контейнеры, как изменяется размер заполненных контейнеров в зависимости от метода шифрования? Почему вы получили такие результаты, обоснуйте

СПИСОК ЛИТЕРАТУРЫ

1. **Авдошин, С. М., Набебин, А. А.** Дискретная математика. Модулярная алгебра, криптография, кодирование. М.: ДМК Пресс, 2017. — 352 с.
2. **Бабаш, А. В., Шанкин, Г. П.** История криптографии. Ч. 1. Москва : Гелиос АРВ, 2002. — 240 с.
3. **Бабаш, А. В., Шанкин, Г. П.** Криптография / под редакцией В.П. Шерстюка, Э.А. Применко. Москва : СОЛОН-Р, 2002. — 512 с.
4. **Бабаш, А. В.** Криптографические и теоретико-автоматные аспекты современной защиты информации. Том 1. Москва : Изд. центр ЕАОИ, 2009. — 414 с.
5. **Бабаш, А. В., Баранова, Е. К.** Криптографические методы защиты информации: учебник. Москва : КноРус, 2016. — 190 с.
6. **Бабаш, А. В., Баранова, Е. К., Ларин, Д.** Информационная безопасность. История специальных методов криптографической деятельности. Москва : ИНФРА-М, РИОР, 2019. — 239 с.
7. **Баричев, С. Г.** Основы современной криптографии / С.Г. Баричев, В.В. Гончаров, Р.Е. Серов. М.: СИНТЕГ, 2011. — 176 с.
8. **Герман, О. Н.** Теоретико-числовые методы в криптографии / О.Н. Герман, Ю.В. Нестеренко. М.: Академия, 2012. — 272 с.
9. **Грибунин, В. Г., Оков, И. Н., Туринцев, И. В.** Цифровая стеганография. Москва : СОЛОН — Пресс, 2002. — 272 с.
10. **Гурин, А. В.** Технологии встраивания цифровых водяных знаков в аудиосигнал / А.В. Гурин, А.А. Жарких, В.Ю. Пластунов. М.: Горячая линия — Телеком, 2015. — 116 с.
11. **Зубов, А. Ю.** Криптографические методы защиты информации. Москва : Гелиос АРВ, 2005. — 192 с.
12. **Зубов, А. Н.** Математика кодов аутентификации / А.Н. Зубов. М.: Гелиос АРВ, 2007. — 288 с.
13. **Конахович, Г. Ф., Пузыренко, А.Ю.** Компьютерная стеганография. Теория и практика. Москва : МК — Пресс, 2006. — 286 с.
14. **Коутинхо, С.** Введение в теорию чисел. Алгоритм RSA. М.: Постмаркет, 2001. — 328 с.
15. **Рябко, Б. Я.** Криптографические методы защиты информации: Учебное пособие для вузов / Б.Я. Рябко, А.Н. Фионов. М.: Горячая линия — Телеком, 2014. — 229 с.
16. **Смарт, Н.** Криптография. Москва : Техносфера, 2006. — 528 с.

17. **Черемушкин, А. В.** Лекции по арифметическим алгоритмам в криптографии. Москва : МЦНМО, 2002. — 104 с.
18. **Шеннон, К.** Работы по теории информации и кибернетике. М.: ИЛ, 1963. — 830 с. (Раздел — Теория связи в секретных системах.)
19. **Шнайер, Б.** Прикладная криптография. Протоколы, алгоритмы, исходные тексты на языке Си: моногр. / Брюс Шнайер. М.: Триумф, 2012. — 816 с.

ТЕСТОВЫЕ ЗАДАНИЯ

1. Всякий источник сообщений можно моделировать списком допустимых (т.е. встречающихся в каких-либо текстах) k -грамм при $k=1,2,3,\dots$ Какие из приведенных k -грамм не являются допустимыми в русском языке? (несколько верных ответов)

- 1) “ШЕЕ”;
- 2) “ЖЫФ”;
- 3) “АУ”;
- 4) “ЮЪХ”;
- 5) “ЖЪН”.

2. Криптограмма получена в результате простой замены:

«ВГАДЮБКГЖЯАО МЮБ ЕДБЕБЗ ЛЖФАЮТ АБЯБГЫЖРАА»

Ключ-подстановка:

А	Б	В	Г	Д	Е	Ж	З	И	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я
Ж	З	Х	К	И	Ц	Ч	Л	А	В	Ъ	Ы	Ь	Б	Д	Г	Е	Ю	Э	Я	П	Р	У	С	Ф	Ш	Т	Щ	М	Н	О

Восстановленный исходный текст:

- 1) «КРИПТОЛОГИЯ ЭТО НАУКА О ЗАЩИТЕ ИНФОРМАЦИИ»;
- 2) «КРИПТОГРАФИЯ ЭТО СПОСОБ ЗАЩИТЫ ИНФОРМАЦИИ»;
- 3) «КРИПТОГРАФИЧЕСКИЕ МЕТОДЫ ЗАЩИТЫ ИНФОРМАЦИИ».

3. Что означает термин «многократное шифрование» применительно к блочным шифрам?

- 1) повторное применение алгоритма шифрования к зашифрованному тексту с теми же ключами;
- 2) шифрование одного и того же блока открытого текста несколько раз с несколькими ключами;
- 3) увеличение числа этапов шифрования открытого текста.

4. Гаммирование чаще всего осуществляется: (несколько верных ответов)

- 1) по модулю 2, если открытый текст представляется в виде бинарной последовательности;
- 2) по модулю 256, если открытый текст представляется в виде последовательности байтов;
- 3) по модулю 16, если открытый текст представлен в цифровом виде;
- 4) по модулю 10, если открытый текст представлен в виде последовательности цифр, что иногда делается в ручных системах шифрования.

5. Основой построения большинства поточных шифров являются:

- 1) генераторы псевдослучайных чисел, в частности, различные комбинации регистров сдвига;
- 2) схемы суммирования по mod 16;
- 3) таблицы подстановок.

6. Зашифрованный методом перестановки открытый текст: «Сертификаты ключей ЭЦП», при ключе длиной 7, и перестановке: {4132756}, имеет вид:

- 1) тСреиифыктал кйюецЦ Э П ;
- 2) юклчТи ЭСЦ еиртфаикт ы ;
- 3) чКилют рСекиафиЭтПы Ц .

7. Зашифровать слово «выборочность» методом перестановки с ключом {3142}:

- 1) бвоычрнотоеьс ;
- 2) ьовбрчоонсьт ;
- 3) ьвброончотось .

8. Зашифровать открытый текст — «field» методом Виженера, ключ — «тооп» (алфавит — латиница):

- 1) gwsup
- 2) gwsyp
- 3) gvsyр

9. Частотный анализ может эффективно применяться для дешифрования шифров:

- 1) перестановки;
- 2) многоалфавитной замены;
- 3) простой замены.

10. Какие меры практической стойкости шифра относительно метода криптоанализа вы можете выделить: (несколько верных ответов)

- 1) вероятность дешифрования за время, не превосходящее T;
- 2) среднее время, необходимое для дешифрования шифра;
- 3) скорость дешифрования шифра.

11. Какие шифры можно называть имитостойкими?

- 1) шифры, обладающие свойством противостоять разрастанию ошибок при расшифровании текстов;
- 2) шифры, обладающие свойством противостоять попыткам навязывания ложной информации.

12. Какие шифры можно называть помехоустойчивыми?

- 1) шифры, обладающие свойством противостоять разрастанию ошибок при расшифровании текстов;
- 2) шифры, обладающие свойством противостоять попыткам навязывания ложной информации.

13. Разрастание числа ошибок означает что

- 1) ошибка в одной букве, допущенная при шифровании, приводит к большому числу ошибок в расшифрованном тексте;
- 2) ошибка в одной букве, допущенная при расшифровании, приводит к последующим ошибкам.

14. Шифр считается совершенным,

- 1) если он не поддается дешифрованию;
- 2) если положение противника, стремящегося к его дешифрованию, не облегчается в результате перехвата зашифрованного текста;
- 3) если требуются большие затраты или мала вероятность успеха его дешифрования.

15. Шифр считается практически стойким,

- 1) если он не поддается дешифрованию;
- 2) если положение противника, стремящегося к его дешифрованию, не облегчается в результате перехвата зашифрованного текста;
- 3) если требуются большие затраты или мала вероятность успеха его дешифрования.

16. Степень неоднозначности восстановления открытого текста при дешифровании

- 1) возрастает при уменьшении материала;
- 2) снижается при уменьшении материала.

17. Какова длина ключа в отечественном стандарте симметричного шифрования:

- 1) 56 бит;
- 2) 124 бит;
- 3) 256 бит

18. Что позволяет предотвратить использование криптографических систем с открытым ключом?

- 1) отказ от информации;
- 2) передачу секретного ключа получателю;
- 3) передачу открытого ключа получателю;
- 4) использование алгоритмов асимметричного шифрования.

19. Ниже перечислены механизмы защиты АИС от несанкционированного доступа и взлома. Что здесь лишнее?

- 1) идентификация и аутентификация пользователей и субъектов доступа;
- 2) управление доступом;
- 3) обеспечение постоянного числа пользователей сети;
- 4) обеспечения целостности.

20. Какова длина блока алгоритма шифрования DES?

- 1) 16 бит;
- 2) 56 бит;
- 3) 64 бита;
- 4) 5 байт.

21. Сколько всего циклов выполняется операция зашифровывания в алгоритме шифрования DES?

- 1) 10;
- 2) 14;
- 3) 16;
- 4) 20.

22. Какой алгоритм не используется при симметричном шифровании?

- 1) поточное шифрование;
- 2) шифрование методом гаммирования;
- 3) блочное шифрование;
- 4) алгоритм Эль-Гамала.

23. Что является преимуществом симметричного шифрования?

- 1) скорость выполнения криптографических преобразований;
- 2) легкость внесения изменений в алгоритм шифрования;
- 3) секретный ключ известен только получателю информации и первоначальный обмен не требует передачи секретного ключа;
- 4) применение в системах аутентификации (электронная цифровая подпись).

24. Длина раундового ключа в отечественном стандарте симметричного шифрования:

- 1) 8 бит;
- 2) 32 бита;
- 3) 48 бит.

25. Как иначе называется асимметричное шифрование?

- 1) шифрование с закрытым ключом;
- 2) шифрование методом Бейтса;

- 3) шифрование с открытым ключом;
- 4) шифрование с переменным ключом.

26. Что является преимуществом асимметричного шифрования?

- 1) скорость выполнения криптографических преобразований;
- 2) легкость внесения изменений в алгоритм шифрования;
- 3) секретный ключ известен только получателю информации и первоначальный обмен не требует передачи секретного ключа;
- 4) применение в системах аутентификации (электронная цифровая подпись).

27. В асимметричных алгоритмах шифрования ключи зашифровывания и расшифровывания всегда:

- 1) разные, хотя и связанные между собой;
- 2) разные, никак не связанные между собой;
- 3) совпадают;
- 4) ключ зашифровывания представляет собой ключ расшифровывания, записанный в обратном порядке.

28. Безопасность системы RSA основана на:

- 1) трудности задачи разложения на простые множители;
- 2) комбинации символов, выбранных случайным образом;
- 3) использовании секретного ключа для шифрования;
- 4) использовании простого делителя в качестве открытого ключа.

29. Что лежит в основе криптографического контроля целостности:

- 1) правильная полная архивация;
- 2) хэш-функция;
- 3) кодовая защита;
- 4) биометрическая идентификация.

30. Хэш-функция используется для (несколько верных ответов):

- 1) для создания сжатого образа сообщения, применяемого в ЭЦП;
- 2) для быстрой передачи данных;
- 3) для идентификации отправителя;
- 4) для защиты пароля;
- 5) для построения кода аутентификации сообщений.

31. Какую роль выполняет электронная цифровая подпись

- 1) информация о передаваемых данных;
- 2) роль обычной подписи в электронных документах;
- 4) дополнительная информация;
- 5) обратный адрес.

32. При формировании цифровой подписи по классической схеме отправитель: (несколько правильных ответов)

- 1) применяет к исходному тексту хэш-функцию;
- 2) применяет к исходному тексту идентификатор отправителя;
- 3) дополняет хэш-образ до длины, требуемой в алгоритме создания ЭЦП;
- 4) выполняет максимальное сжатие;
- 5) вычисляет ЭЦП по хэш-образу с использованием секретного ключа создания подписи.

Таблица ответов

Номер вопроса	1	2	3	4	5	6	7	8
Правильный ответ	2, 4, 5	2	2	1, 2, 4	1	1	1	2
Номер вопроса	9	10	11	12	13	14	15	16
Правильный ответ	3	1, 2	2	1	1	2	3	1
Номер вопроса	17	18	19	20	21	22	23	24
Правильный ответ	3	2	3	3	3	4	1	2
Номер вопроса	25	26	27	28	29	30	31	32
Правильный ответ	3	3, 4	1	1	2	1, 4, 5	2	1, 3, 5

Варианты заданий с решениями

Вариант 1

Расшифровать криптограмму, полученную с использованием шифра простой замены, при имеющемся ключе шифрования.

Текст: ВГАДЮБКГЖЯАОМЮБЕДБЕБЗЛЖФАЮТАБЯБГЫЖРАА

Ключ-подстановка:

А	Б	В	Г	Д	Е	Ж	З	И	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я
Ж	З	Х	К	И	Ц	Ч	Л	А	В	Ъ	Ы	Ь	Б	Д	Г	Е	Ю	Э	Я	П	Р	У	С	Ф	Ш	Т	Щ	М	Н	О

Варианты ответов:

1. СПОСОБ ЗАЩИТЫ ИНФОРМАЦИИ ЭТО КРИПТОГРАФИЯ
2. КРИПТОГРАФИЯ БОЛЕЕ НАДЕЖНЫЙ СПОСОБ ЗАЩИТЫ
3. КРИПТОГРАФИЯ ЭТО СПОСОБ ЗАЩИТЫ ИНФОРМАЦИИ

Ответ: 3**Решение:**Запишем **ключ-обратной подстановки** на основе ключа-подстановки:

А	Б	В	Г	Д	Е	Ж	З	И	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я
И	О	К	Р	П	С	А	Б	Д	Г	З	Э	Ю	Я	Х	Ц	Ш	Ы	Ч	Щ	В	Е	Ж	Ъ	Ь	Л	М	Н	У	Т	Ф

При помощи этого ключа расшифровываем текст. В соответствии с ключом-обратной подстановки первая буква зашифрованного текста «В» перейдет в «К», «Г» перейдет в «Р» и так далее. В итоге получим расшифрованный текст:

КРИПТОГРАФИЯ ЭТО СПОСОБ ЗАЩИТЫ ИНФОРМАЦИИ**Вариант 2**

Расшифровать криптограмму: 13 34 22 24 44 34 15 42 22 34 43 45 32.

Известно, что криптограмма получена с использованием квадрата Полибия:

A	B	C	D	E
F	G	H	I,J	K
L	M	N	O	P
Q	R	S	T	U
V	W	X	Y	Z

Варианты ответов:

1. ERGO SUM COGITO
2. COGITO ERGO SUM
3. COGITO SUM ERGO

Ответ: 2**Решение:**

Нумеруем строки и столбцы квадрата Полибия:

	1	2	3	4	5
1	A	B	C	D	E
2	F	G	H	I,J	K
3	L	M	N	O	P
4	Q	R	S	T	U
5	V	W	X	Y	Z

Буквы исходного текста находятся на пересечении соответствующей строки и столбца квадрата, например, 13 (строка 1, столбец 3 — буква исходного текста «С») и т.д.

В итоге, получаем расшифрованный текст: COGITO ERGO SUM (лат. «Я мыслю, следовательно, существую», Р. Декарт).

Вариант 3

Дана таблица замен:

	1	2	3	4	5	6
1	А	Б	В	Г	Д	Е
2	Е	Ж	З	И	Й	К
3	Л	М	Н	О	П	Р
4	С	Т	У	Ф	Х	Ц
5	Ч	Ш	Щ	Ъ	Ы	Ь
6	Э	Ю	Я			

Известно, что буква «О» в исходном тексте, заменена числом 34 в криптограмме.

Какая из криптограмм соответствует исходному тексту: КРИПТОГРАФИЯ?

Варианты ответов:

1. 26 36 24 35 42 34 41 24 41 42 16 63
2. 26 36 24 35 42 34 14 36 11 32 32 11
3. 26 36 24 35 42 34 14 36 11 44 24 63

Ответ: 3

Вариант 4

Зашифрован исходный текст: FIELD методом Виженера, ключ (лозунг): KEY (алфавит — латиница 25 символов, исключена буква «W»). Какая из криптограмм верная?

Варианты ответов:

1. PXCvH
2. PMSvH
3. RMCyH

Ответ: 2

Решение:

Исходный текст: F I E L D

Лозунг (с повторением по длине исходного текста): K E Y K E

Буква исходного текста — строка; буква лозунга — столбец. На пересечении строки-**F** и столбца-**K** находим букву зашифрованного текста: «**P**», и так далее.

Правильный ответ:

PMSCVN

A B C D E F G H I J K L M N O P Q R S T U V X Y Z

A A B C D E F G H I J K L M N O P Q R S T U V X Y Z
B B C D E F G H I J K L M N O P Q R S T U V X Y Z A
C C D E F G H I J K L M N O P Q R S T U V X Y Z A B
D D E F G H I J K L M N O P Q R S T U V X Y Z A B C
E E F G H I J K L M N O P Q R S T U V X Y Z A B C D
F F G H I J K L M N O P Q R S T U V X Y Z A B C D E
G G H I J K L M N O P Q R S T U V X Y Z A B C D E F
H H I J K L M N O P Q R S T U V X Y Z A B C D E F G
I I J K L M N O P Q R S T U V X Y Z A B C D E F G H
J J K L M N O P Q R S T U V X Y Z A B C D E F G H I
K K L M N O P Q R S T U V X Y Z A B C D E F G H I J
L L M N O P Q R S T U V X Y Z A B C D E F G H I J K
M M N O P Q R S T U V X Y Z A B C D E F G H I J K L
N N O P Q R S T U V X Y Z A B C D E F G H I J K L M
O O P Q R S T U V X Y Z A B C D E F G H I J K L M N
P P Q R S T U V X Y Z A B C D E F G H I J K L M N O
Q Q R S T U V X Y Z A B C D E F G H I J K L M N O P
R R S T U V X Y Z A B C D E F G H I J K L M N O P Q
S S T U V X Y Z A B C D E F G H I J K L M N O P Q R
T T U V X Y Z A B C D E F G H I J K L M N O P Q R S
U U V X Y Z A B C D E F G H I J K L M N O P Q R S T
V V X Y Z A B C D E F G H I J K L M N O P Q R S T U
X X Y Z A B C D E F G H I J K L M N O P Q R S T U V
Y Y Z A B C D E F G H I J K L M N O P Q R S T U V X
Z Z A B C D E F G H I J K L M N O P Q R S T U V X Y

Вариант 5

Расшифровать криптограмму, полученную с помощью метода случайного гаммирования, считая, что буквы алфавита пронумерованы от 0 до 32, соответственно. Зная определенную гамму.

А	Б	В	Г	Д	Е	Ё	Ж	З	И	Й	К	Л	М	Н	О	П
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я	
17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	

Текст: ХСЩРАБЛЮТЕЧЙ

Гамма(ключ):

11	1	17	1	14	19	9	14	19	17	15	11
----	---	----	---	----	----	---	----	----	----	----	----

Варианты ответов:

1. КРИПТОГРАФИЯ
2. КРИПТОЗАЩИТА
3. КАРДИОГРАФИЯ

Ответ: 1**Решение:**

Y	X	C	Щ	P	A	Б	Л	Ю	T	E	Ч	Й
	22	18	26	17	0	1	12	31	19	5	24	10
K	11	1	17	1	14	19	9	14	19	17	15	11
X	11	17	9	16	19	15	3	17	0	21	9	32
	K	P	И	П	T	O	Г	P	A	Ф	И	Я

Первая строка - зашифрованный текст.

Вторая строка — номера букв зашифрованного текста.

Третья строка — гамма.

Четвертая строка — номера букв открытого текста в алфавите.

Пятая строка — открытый текст, в соответствии с номерами.

Вычитаем из номера буквы алфавита соответствующий компонент ключа, если разность меньше 0, то прибавляем 33. Например первая буква ($22-11=11$), а пятая буква ($0-14=-14$; $-14+33=19$). Затем записываем в расшифрованный текст в соответствии с номером в алфавите: КРИПТОГРАФИЯ

Вариант 6

Зашифровать фразу: СООБЩИТЕ О ПРИБЫТИИ ЛИНЕЙНЫХ КОРАБЛЕЙ шифром вертикальной перестановки (фраза для шифрования записывается слитно без пробелов). В качестве секретного ключа (лозунга) используется фраза: БОЖЕ ЦАРЯ ХРАНИ, фраза записывается слитно, и буквы нумеруются в алфавитном порядке (при этом, если буква встречается несколько раз, номера ей присваиваются последовательно):

Б	О	Ж	Е	Ц	А	Р	Я	Х	Р	А	Н	И
3	8	5	4	12	1	9	13	11	10	2	7	6

Варианты ответов:

1. ИЕИРЫСЫЕБИБОИАБКИХОТРТНЙПНОЙЩЛЛЕЕ
2. ЕЕИРЫСЫОБИБОИАБКИХОТРТНЙПНОЙЩЛЛИИ
3. ИИЕРЫСЫОБИБОИАБКИХОТРТНЙПНОЙЩЛЛЕЕ

Ответ: 3**Решение:**

Ключевая последовательность является так называемым номерным рядом. Записываем исходный текст без пробелов: СООБЩИТЕ-ОПРИБЫТИИЛИНЕЙНЫХКОРАБЛЕЙ. При шифровании выписывается номерной ряд, а под ним шифруемый текст:

3	8	5	4	12	1	9	13	11	10	2	7	6
С	О	О	Б	Щ	И	Т	Е	О	П	Р	И	Б
Ы	Т	И	И	Л	И	Н	Е	Й	Н	Ы	Х	К
О	Р	А	Б	Л	Е	Й						

Шифрование производится выпиской по колонкам номерного ряда. В первой колонке стоят буквы ИИЕ, во второй РЫ, в третьей СЫО и т.д.

Получим шифрованный текст:

ИИЕРЫСЫОБИБОИАБКХОТРТНЙПНОЙЩЛЛЕЕ

Вариант 7

Зашифровать открытый текст: ШИФРОВАНИЕ_ПЕРЕСТАНОВКОЙ, используя метод усложненной перестановки. Запись исходного текста по строкам некоторой матрицы, ключ $k_1 \{5-3-1-2-4-6\}$, чтение зашифрованного текста по столбцам этой матрицы ключ $k_2 \{4-2-3-1\}$.

Варианты ответов:

1. ПСНОРЙЕРВАИК_ЕАНФОИЕОТШВ
2. ПСНОРЙЕРВА_ИКЕАНФОИЕОТШВ
3. ЕАНФОИЕОТШВ_ПСНОРЙЕРВАИК

Ответ: 1**Решение:**

1	И	Е	—	П
2	Е	Р	Е	С
3	О	В	А	Н
4	Т	А	Н	О
5	Ш	И	Ф	Р
6	В	К	О	Й
k_1/k_2	1	2	3	4

Открытый текст: «ШИФРОВАНИЕ_ПЕРЕСТАНОВКОЙ».

Матрица из четырех столбцов. Ключи: $k_1 \{5-3-1-2-4-6\}$; $k_2 \{4-2-3-1\}$.

Запись по строкам производится в соответствии с ключом k_1 .

Чтение по столбцам в соответствии с ключом k_2 .

Шифротекст: «ПСНОРЙЕРВАИК_ЕАНФОИЕОТШВ».

Вариант 8

Расшифровать криптограмму, полученную с использованием шифра перестановки, имея ключ.

Криптограмма: ПОРИКТФЧРАГИА СКЕЯИААЩЗТ

Ключ:

1	2	3	4	5	6
5	3	4	1	6	2

Варианты ответов:

1. КРИПТОЗАЩИТА ИНФОРМАЦИИ
2. КРИПТОГРАФИЧЕСКАЯ ЗАЩИТА
3. ЗАЩИТА КРИПТОГРАФИЧЕСКАЯ

Ответ: 2

Решение:

Составим **ключ-обратной подстановки**:

1	2	3	4	5	6
4	6	2	3	1	5

Разобьём текст в соответствии с длиной ключа и запишем в таблицу:

П	О	Р	И	К	Т
Ф	Ч	Р	А	Г	И
А		С	К	Е	Я
И	А	А	Щ	З	Т

Переставим столбцы в соответствии с ключом-обратной подстановки. Первый столбец станет четвертым, второй — шестым, и так далее. После перестановки получим таблицу:

К	Р	И	П	Т	О
Г	Р	А	Ф	И	Ч
Е	С	К	А	Я	
З	А	Щ	И	Т	А

Затем выпишем строки по порядку и получим расшифрованный текст:

КРИПТОГРАФИЧЕСКАЯ ЗАЩИТА

